

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОРЛОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ И.С. ТУРГЕНЕВА»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по направлению подготовки
09.03.04 Программная инженерия
Направленность (профиль) Индустриальное производство программного
обеспечения

Студента Банных Марии Алексеевны шифр 220885
Институт приборостроения, автоматизации и информационных технологий
Кафедра информационных систем и цифровых технологий

Тема выпускной квалификационной работы

«Разработка онлайн-сервиса для ориентирования по корпусам университета на
базе WebGL»

Студент	М.А. Банных
Руководитель	А.Ю. Ужаринский
Нормоконтроль	О.В. Захарова
И.о. зав. кафедрой информационных систем и цифровых технологий	Д.В. Рыженков

Орёл 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОРЛОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ И.С. ТУРГЕНЕВА»

Институт приборостроения, автоматизации и информационных технологий
Кафедра информационных систем и цифровых технологий
Направление 09.03.04 Программная инженерия
Направленность (профиль) Индустриальное производство программного обеспечения

УТВЕРЖДАЮ:

И.о. зав. кафедрой

_____ Д.В. Рыженков

«__» _____ 20__ г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

студента Банных Марии Алексеевны

шифр 220885

1 Тема ВКР «Разработка онлайн-сервиса для ориентирования по корпусам университета на базе WebGL»

Утверждена приказом по университету от «26» ноября 2025г. № 8-305

2 Срок сдачи студентом законченной работы «22» мая 2026г.

3 Исходные данные к работе

Теоретический материал; информация о предметной области

4 Содержание пояснительной записки (перечень подлежащих разработке вопросов)

Анализ задачи и формулировка требований

Анализ требований и определение спецификаций

Проектирование онлайн-сервиса

Реализация и тестирование онлайн-сервиса

5 Перечень демонстрационного материала

Презентация, отображающая основные этапы и результаты выполнения ВКР

Дата выдачи задания « 26 » ноября 2025 г.

Руководитель _____ А.Ю. Ужаринский
(подпись)

Задание принял к исполнению _____ М.А. Банных
(подпись)

КАЛЕНДАРНЫЙ ПЛАН

Наименование этапов работы	Сроки выполнения этапов работы	Примечание
Анализ задачи	26.11.2025 - 17.12.2025	
Определение требований и спецификаций	18.12.2025 - 14.01.2026	
Проектирование	15.01.2026 - 17.03.2026	
Реализация	18.03.2026 - 11.05.2026	
Тестирование	12.05.2026 - 18.05.2026	
Оформление ВКР	19.05.2026 - 22.05.2026	

Студент _____ М.А. Банных
(подпись)

Руководитель _____ А.Ю. Ужаринский
(подпись)

АННОТАЦИЯ

ВКР 86 с., 42 рис., 12 табл., 13 источников, 1 прил.

ИНФОРМАЦИОННАЯ СИСТЕМА, НАВИГАЦИОННАЯ СИСТЕМА, ОРИЕНТИРОВАНИЕ, 3D ВИЗУАЛИЗАЦИЯ.

Выпускная квалификационная работа посвящена разработке онлайн-сервиса для ориентирования по корпусам университета на базе WebGL.

В первой главе была проанализирована предметная область, произведён анализ существующих методов решения и аналогов навигационной системы. В результате были сформированы требования к онлайн-сервису и средствам разработки.

Во второй главе был произведён анализ требований, разработана обобщённая архитектура, определены функциональная, информационная и поведенческая спецификации.

В третьей главе была спроектирована структура онлайн-сервиса, компоненты и структура данных. Также были разработаны алгоритмы и интерфейс.

В четвертой главе была представлена реализация компонентов и интерфейса, показаны экранные формы, проведено тестирование и отладка онлайн-сервиса.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 АНАЛИЗ ЗАДАЧИ И ФОРМУЛИРОВКА ТРЕБОВАНИЙ К РАЗРАБОТКЕ.....	6
1.1 Анализ существующих методов и формализация задачи по разработке онлайн-сервиса	6
1.2 Обзор аналогичных решений	11
1.3 Формулировка функциональных и нефункциональных требований	19
1.4 Обоснование и формулировка средств разработки онлайн-сервиса.....	20
2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	22
2.1 Разработка обобщённой архитектуры.....	22
2.2 Разработка функциональных спецификаций	23
2.3 Разработка информационных спецификаций	25
2.4 Разработка поведенческих спецификаций.....	26
3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	31
3.1 Проектирование структуры	31
3.2 Проектирование компонентов.....	33
3.3 Проектирование структур данных	38
3.4 Проектирование алгоритмов	42
3.5 Проектирование интерфейсов	50
4 РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ОТЛАДКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	53
4.1 Реализация компонентов программного обеспечения	53
4.2 Реализация интерфейса пользователя	55
4.3 Тестирование и отладка программного обеспечения.....	58
ЗАКЛЮЧЕНИЕ	71
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	72
Приложение – Фрагменты исходного кода	74
УДОСТОВЕРЯЮЩИЙ ЛИСТ № 220885.....	84
ИНФОРМАЦИОННО-ПОИСКОВАЯ ХАРАКТЕРИСТИКА ДОКУМЕНТА НА ЭЛЕКТРОННОМ НОСИТЕЛЕ	85

ВВЕДЕНИЕ

Цифровизация в настоящее время охватывает всё большее количество сфер жизни человека. Это не обошло стороной и картографию. Ещё пол века назад для ориентирования в городе или стране повсеместно применялись бумажные карты. Но в современном мире им на смену пришли электронные карты. Они имеют множество преимуществ: компактность (размещены в телефоне или компьютере), наличие встроенной навигации, поиска и других функций, повышающих удобство использования.

Однако необходимость в ориентировании возникает не только между зданиями, но и внутри них. Если не принимать во внимание планы эвакуации, то навигация чаще всего осуществляется при помощи указателей. Такой подход малоэффективен в больших архитектурных сооружениях со сложной планировкой. В связи с этим возникает необходимость в более понятных, быстрых и точных методов внутренней навигации. Это особенно актуально для зданий, предназначенных для большого количества людей, состав которых периодически меняется.

Темой выпускной квалификационной работы является разработка онлайн-сервиса для ориентирования по корпусам университета на базе WebGL.

Целью выпускной квалификационной работы является снижение временных затрат на поиск аудиторий и востребованных помещений, к которым относятся буфет, столовая, туалет, библиотека и т.д.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- проанализировать задачу и сформулировать требования к разработке;
- проанализировать требования и определить спецификации программного обеспечения;
- спроектировать онлайн-сервис;
- реализовать, протестировать и отладить онлайн-сервис.

1 АНАЛИЗ ЗАДАЧИ И ФОРМУЛИРОВКА ТРЕБОВАНИЙ К РАЗРАБОТКЕ

1.1 Анализ существующих методов и формализация задачи по разработке онлайн-сервиса

Ориентирование внутри корпуса необходимо для быстрого поиска аудитории студентами, преподавателями или гостями университета.

Для новых студентов, только поступивших в ВУЗ, структура корпуса может показаться очень сложной и запутанной особенно если он имеет сложную архитектуру. В связи с этим им приходится либо постоянно обращаться за помощью к окружающим, либо самостоятельно искать нужные аудитории, что может занимать довольно большое количество времени. Иностранным студентам, столкнувшимся с языковым барьером, эта задача даётся ещё труднее: у них остаётся только вариант самостоятельного поиска аудитории.

Многие преподаватели университета проводят занятия в разных корпусах, перемещаясь между ними в течение дня, тратя на это значительное количество времени. В связи с этим для них очень важно хорошо ориентироваться во всех зданиях, где у них будут проводиться пары, чтобы не тратить оставшееся время на поиск нужной аудитории. Однако, учитывая разнообразие архитектуры учебных корпусов, преподавателям приходится запоминать множество деталей о расположении помещений, что сильно увеличивает затрачиваемое время.

Студенты из разных факультетов, а также гости, посещающие университет в рамках различных мероприятий, тоже могут столкнуться трудностями при поиске нужных помещений. Незнание расположения аудиторий, актовых залов, выставочных зон может привести к опозданиям, что может негативно сказаться на общем впечатлении о мероприятии.

Абитуриенты, рассматривающие университет в качестве будущего учебного заведения, иногда проявляют интерес к структуре корпусов, в которых они будут обучаться. Поэтому для университета важно дать возможность ознакомиться с ней заранее, чтобы абитуриент чувствовал себя более уверенно при поступлении.

Все посетители корпуса, должны иметь чёткое представление о расположении наиболее востребованных помещений, таких как библиотека, буфет, столовая, туалет и т.д. Это позволит им быстрее находить нужные кабинеты, экономя время, и чувствовать комфорт от пребывания в учебном корпусе, что очень важно для положительного восприятия университета как современного и удобного учебного пространства.

Подводя итоги, можно сказать, что ориентирование в корпусах университета по-прежнему остаётся проблемой, что обусловлено следующими факторами:

- большим количеством корпусов;
- сложной архитектурой зданий;
- отсутствием удобной системы навигации.

Эти факторы приводят к возникновению таких проблем как:

- опоздания;
- отрицательное восприятие университета как учебного пространства;
- ухудшение мотивации учиться;
- сложность в адаптации.

Из-за отсутствия эффективной системы навигации студентам и преподавателям приходится использовать различные методы ориентирования, каждый из которых требует значительных затрат по времени. Способы представлены на диаграмме, изображённой на рисунке 1.1.1.

Первый способ – это самостоятельный поиск нужного помещения. В этом случае студент или преподаватель опираются на свой опыт ориентирования в других корпусах или зданиях университета. Например, они предполагают, что кабинеты, начинающиеся с цифры 1, расположены на первом этаже, кабинеты, начинающиеся с 2 – на втором этаже и так далее. В некоторых случаях кабинеты, начинающиеся с одной и той же цифры, могут находиться на одном этаже, но в разных коридорах, ведущих в разные направления. Это значительно усложняет поиск, поскольку может потребоваться зайти в несколько различных ответвлений, проверяя, не находится ли там искомое помещение, что сильно увеличивает время поиска. А иногда может оказаться так, что номера кабинетов не следуют

установленной логике, где первая цифра отвечает за номер этажа, тогда самостоятельный поиск становится малоэффективным и почти невозможным.

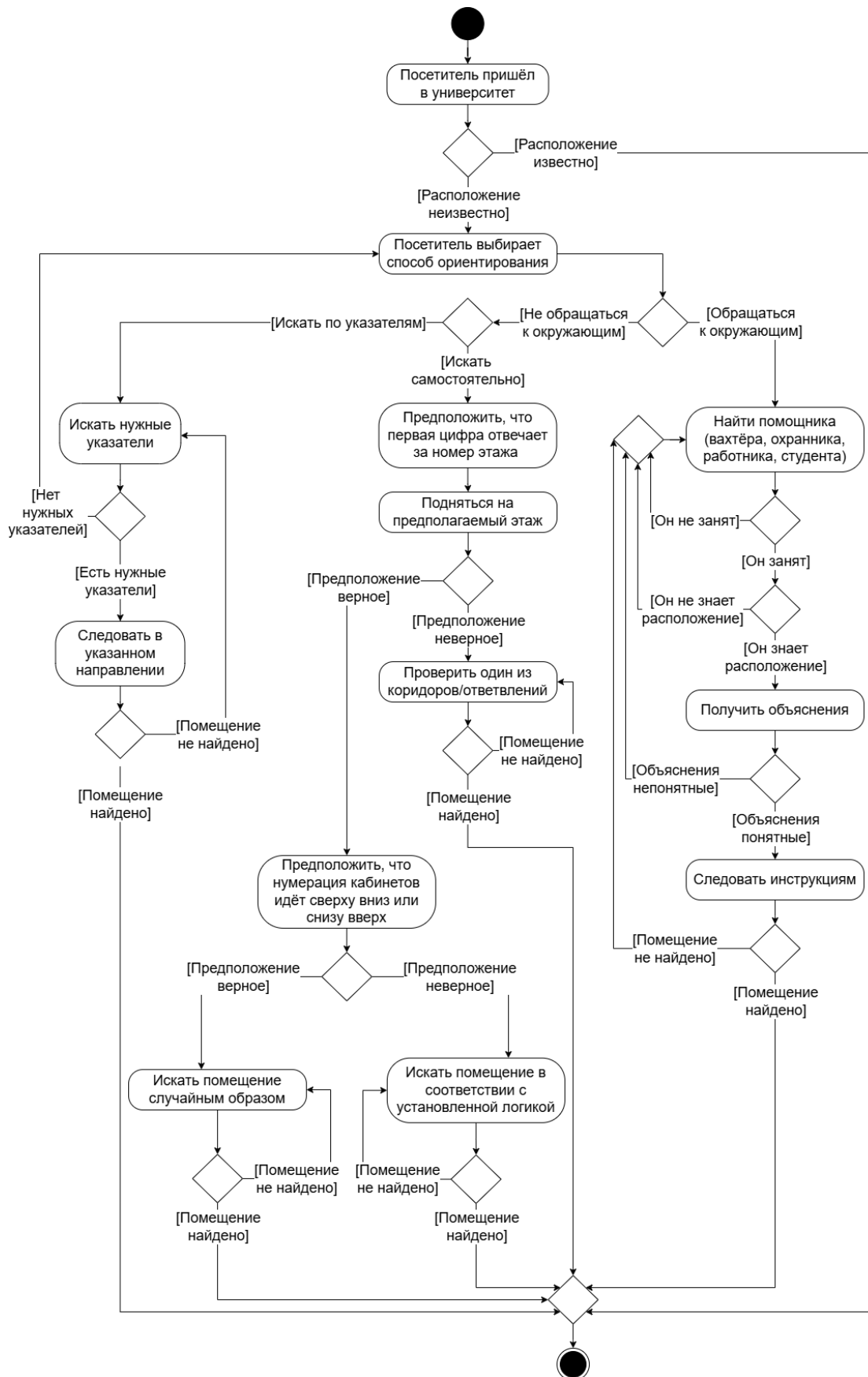


Рисунок 1.1.1 – Описание существующих подходов к поиску нужного помещения

Второй способ – обращение за помощью к вахтёру, студентам, преподавателям или другим, находящимся в корпусе лицам. Несмотря на то, что этот способ кажется быстрым, у него есть свои недостатки. Объяснения, которые дают окружающие, могут быть достаточно запутанными, поскольку инструкции могут включать множество шагов и деталей. Необходимость постоянно держать в голове последовательность действий, считать проходы и лестницы значительно усложняет поиск и увеличивает риск заблудиться или потратить много времени. Кроме того, человек, к которому обращаются за помощью, может не обладать точной информацией о расположении нужного помещения, а в некоторых ситуациях он сам может спешить или быть занят. В таком случае ему будет трудно уделить время и подробно объяснить путь.

Третий способ – ориентирование по указателям, размещённым в университете, как показано на рисунке 1.1.2. Недостатком данного метода является то, что указатели внутри корпусов ведут только к некоторым объектам, таким как лифт, буфет или другие востребованные помещения. Кроме того, они могут временно размещаться при проведении специальных мероприятий, чтобы гости могли сориентироваться и добраться до нужного места. В результате стрелки и указатели помогают найти наиболее важные точки, однако по ним нельзя определить расположение каждого кабинета, поскольку они не предназначены для отображения всей внутренней планировки.



Рисунок 1.1.2 – Ориентирование по указателям

Все перечисленные способы требуют значительных временных затрат у студентов и преподавателей, что может привести к опозданиям на занятия. Поэтому возникает необходимость в разработке онлайн-сервиса, который будет предоставлять трёхмерные модели корпусов с системой навигации, что позволит быстро и точно ориентироваться внутри них. Именно 3D-модели позволяют создать более наглядные и удобные в использовании инструменты для ориентирования внутри зданий. Исследования показывают, что трёхмерные карты значительно улучшают эффективность навигации, особенно в сложных строениях, где 2D-схемы могут быть недостаточно информативными. Кроме того, 3D-модели вызывают больший интерес у пользователей и лучше запоминаются, а также оставляют хорошее впечатление и демонстрируют технологический уровень ВУЗа.

Сайт университета содержит информацию о местоположении корпусов, но для студентов и преподавателей зачастую более проблематичным является ориентирование именно внутри здания. Онлайн-сервис должен помочь легко находить нужные помещения и быстро прокладывать маршрут между ними, что значительно сократит время, затрачиваемое на поиск. Кроме того, с преподавателей, студентов и гостей будет снята необходимость запоминать структуру большого количества корпусов.

Также, благодаря поддержке интерфейса на русском и английском языках, система станет доступной для широкой аудитории, охватывая как русскоговорящих, так и англоговорящих студентов. Важными элементами навигационной системы станут подсказки и инструкции по ориентированию внутри корпусов, а также возможность вращения, перемещения и масштабирования 3D-моделей корпусов для более детального осмотра. Перечень востребованных помещений обеспечит возможность быстрого построения маршрута до них.

Все эти элементы навигационной системы обеспечат пользователю понятный интерфейс, который позволит взаимодействовать с системой без необходимости долгого освоения или поиска нужных функций. В данном случае под долгим освоением понимается трата более 3 минут на понимание работы основного

интерфейса программы: как выбрать корпус, как построить маршрут, как найти помещение.

1.2 Обзор аналогичных решений

В настоящее время всё больше ВУЗов осознают важность разработки и внедрения систем, которые помогают студентам и сотрудникам ориентироваться в сложной структуре учебных корпусов. В результате, многие университеты создают собственные интерактивные карты, позволяющие ознакомиться с внутренним устройством зданий, расположением аудиторий, лабораторий, административных помещений и других объектов. Такие решения способствуют более быстрой адаптации новых студентов, создают положительное впечатление об учебном заведении и повышают его привлекательность. Кроме того, они служат важным инструментом для повышения интереса потенциальных абитуриентов и работников к университету.

Однако провести полноценный анализ реализованных систем мешает то, что большинство подобных решений находится в закрытом доступе. Чтобы воспользоваться ими, необходимо быть зарегистрированным пользователем, то есть являться студентом или работником этого учреждения. Это сильно ограничивает возможность изучения этих систем и сравнения их между собой.

В связи с этим, будут рассмотрены только те решения, которые доступны для общего пользования.

Для анализа были выбраны следующие системы, предоставляющие возможность осмотра корпуса:

1. Виртуальный тур Орловского государственного университета [1];
2. Интерактивная 2D карта РТУ МИРЭА [2];
3. Интерактивная карта-навигатор по корпусам ОГУ имени И.С. Тургенева [3].

Виртуальный тур Орловского государственного университета – это раздел, созданный для ознакомления с учебными корпусами, общежитиями и административными зданиями. Он предназначен для пользователей, желающих

получить представление о внутреннем устройстве и интерьере университета без необходимости личного посещения. Виртуальная экскурсия позволяет увидеть внутренние помещения и понять атмосферу учебного заведения. Раздел размещён на сайте ОГУ имени И.С. Тургенева, доступен без авторизации и предоставляет возможность для любого желающего «прогуляться» по корпусам и коридорам ВУЗа не выходя из дома.

Виртуальный тур предоставляет возможность выбрать один из корпусов или зданий, принадлежащих ОГУ. При выборе система загружает панораму, с которой можно взаимодействовать, как показано на рисунке 1.2.1. Экран разделён на рабочую область с изображением и панель управления, включающую кнопки навигации: поворота изображения вправо, влево, вверх и вниз, а также функции приближения, отдаления и открытия полноэкранного режима.



Рисунок 1.2.1 – Виртуальный тур ОГУ

Перемещение внутри панорамы осуществляется при помощи стрелок, расположенных прямо на изображении, как показано на рисунке 1.2.2, а также через элементы меню, позволяющие выбрать другую локацию.

Несмотря на преимущества виртуальных экскурсий, существуют и определённые недостатки.

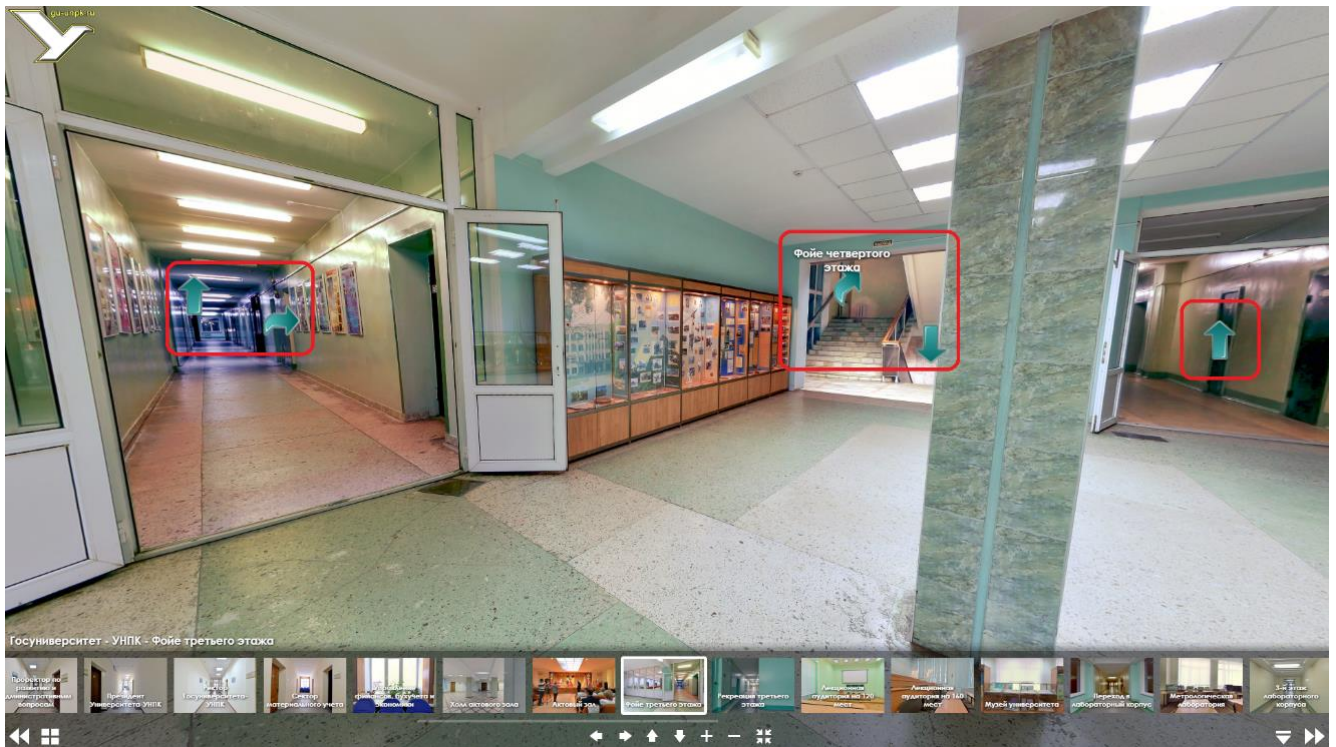


Рисунок 1.2.2 – Перемещение по виртуальному туру ОГУ

Одним из существенных минусов является отсутствие наглядного представления о планировке корпуса, что затрудняет понимание его внутренней структуры. Кроме того, виртуальный тур не позволяет построить конкретный маршрут внутри здания, что особенно важно для новых студентов и гостей, которым необходимо быстро ориентироваться и находить нужные помещения. Поэтому тур позволяет пользователю лишь получить общее впечатление об университете, но не помогает ориентироваться в корпусах.

Российский технологический университет МИРЭА предоставляет своим студентам и сотрудникам интерактивную 2D-карту учебных корпусов. Этот инструмент позволяет получить общее представление о планировке и внутренней структуре зданий, а также служит эффективным средством для ориентирования внутри. Внешний вид 2D-карты представлен на рисунке 1.2.3. Она представляет собой схему здания, которую можно масштабировать и перемещать. Карта также позволяет выбрать один из трёх корпусов университета. На ней можно перемещаться между этажами, выбирая номер в нижней правой части экрана.

В верхней части расположена строка поиска, где можно ввести номер помещения, чтобы быстро его найти. Также предусмотрен просмотр

дополнительной информации, такой как нагрузка аудиторий, наличие свободных помещений и время, для которого отображаются эти параметры.

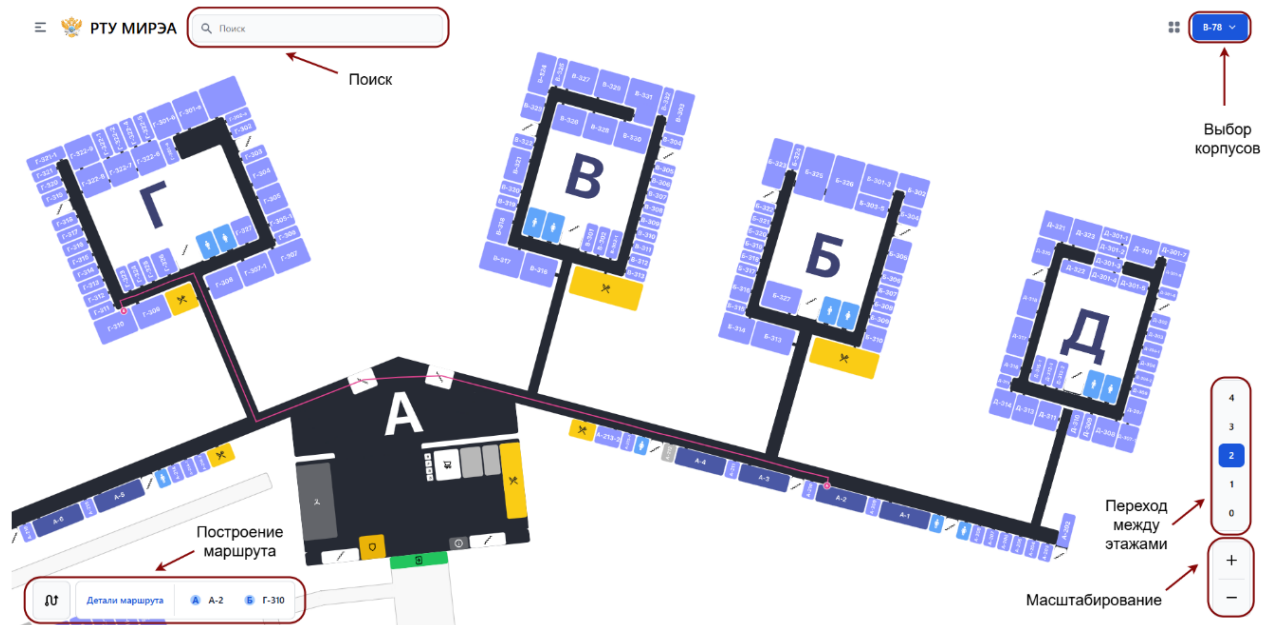


Рисунок 1.2.3 – Интерактивная 2D карта РТУ МИРЭА

Главной функцией интерактивной 2D-карты является возможность проложить маршрут от одного помещения до другого. Для этого необходимо открыть окно построения маршрута, которое изображено на рисунке 1.2.4. Это можно сделать, нажав кнопку в левом нижнем углу экрана. После ввода начальной и конечной точки и нажатия кнопки «построить» карта проложит кратчайший путь и покажет его на схеме.

Для того, чтобы получить краткую информацию о какой-либо аудитории, достаточно просто кликнуть на неё. Кроме того, есть возможность построить маршрут с выбранным помещением в качестве начальной или конечной точки либо просмотреть его расписание.

Интерактивная 2D-карта РТУ МИРЭА является удобным инструментом для ориентирования внутри корпуса и обладает множеством преимуществ. Однако, в сравнении с трехмерной моделью, 2D-схема менее информативна и не передает весь объем пространства.

Исследования показывают, что трехмерные карты значительно повышают эффективность навигации, особенно в сложных зданиях, где 2D схемы могут оказаться недостаточно наглядными [4].

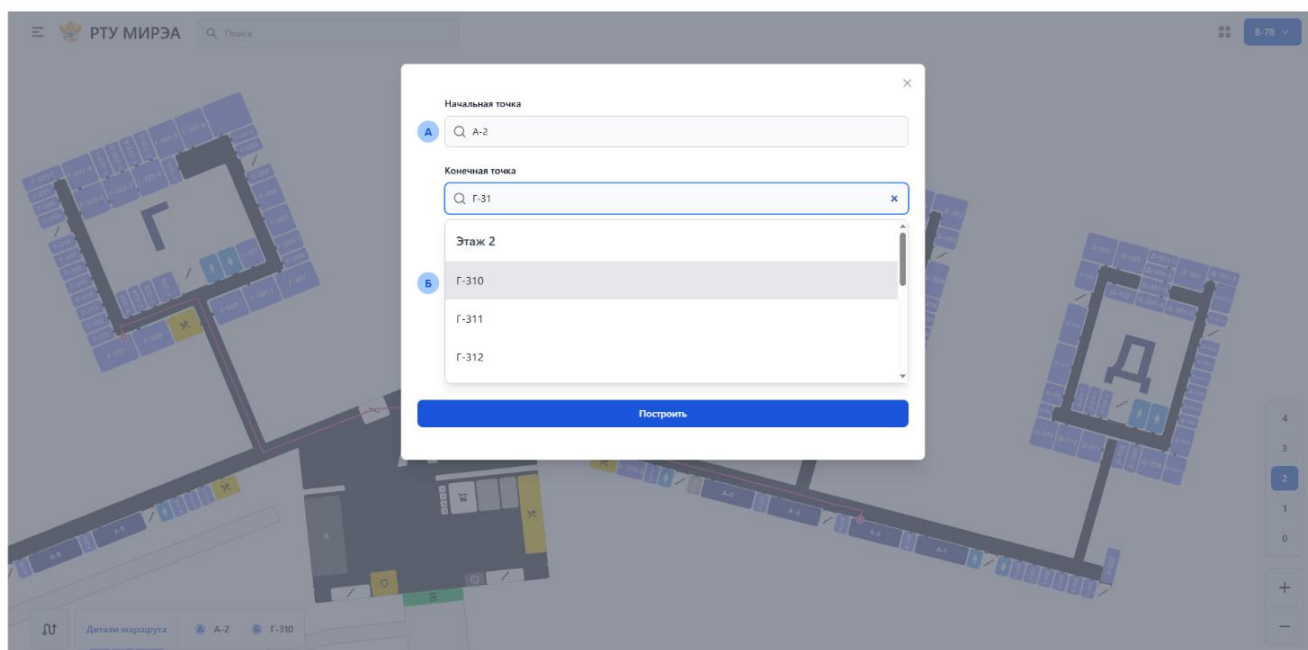


Рисунок 1.2.4 – Построение маршрута с помощью интерактивной 2D карты РТУ МИРЭА

Также известно, что 3D-модели вызывают больший интерес и лучше запоминаются [5].

Интерактивная карта-навигатор по корпусам ОГУ им. И.С. Тургенева представляет собой удобный инструмент для быстрого ориентирования внутри корпусов. Эта 2D карта позволяет пользователю выбрать один из двух доступных учебных корпусов, как показано на рисунке 1.2.5.



Рисунок 1.2.5 – Выбор учебного корпуса

После выбора открывается подробная схема здания, которая позволяет перемещаться по этажам, а также выполнять такие действия, как приближение, отдаление и перемещение по карте для более удобного обзора.

На карте реализована функция поиска помещений, для которой в верхней части интерфейса расположена поисковая строка. Введя в неё название или номер кабинета, пользователь может быстро получить необходимую информацию. Кроме того, предусмотрена возможность построения маршрута от одной точки до другой, что необходимо для быстрого поиска нужного помещения. Кроме того, при выборе кабинета, на схеме отображается минимальная информация о нём. Это позволяет избежать перегруженности данными, сохраняя ясность восприятия. Построение маршрута показано на рисунке 1.2.6.

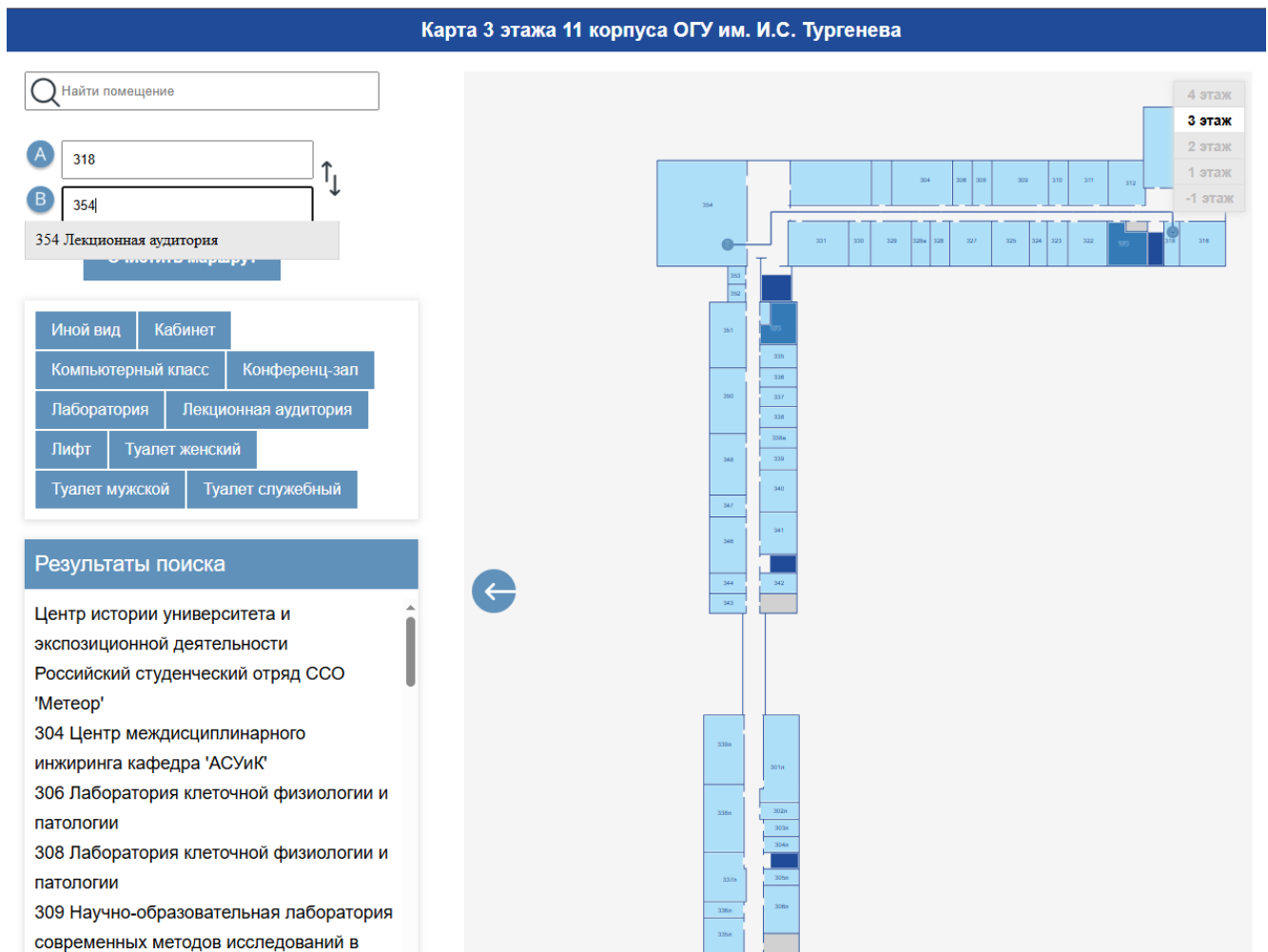


Рисунок 1.2.6 – Построение маршрута с помощью интерактивной карты-навигатора по корпусам ОГУ им. И.С. Тургенева

Особенностью системы является панель с результатами поиска — она представляет собой список кабинетов выбранного этажа. Пользователь может дополнительно настроить эту панель, добавляя фильтры, такие как "учебные кабинеты", "лаборатории" и так далее. Такая гибкая настройка значительно

упрощает навигацию и позволяет быстро ориентироваться в пространстве, выбирая только нужные категории объектов.

Несмотря на все преимущества, у системы есть и определённые недостатки. В данный момент в ней представлены только два корпуса из всех учебных зданий ОГУ, что ограничивает её полноту и функциональность. Кроме того, система ещё не доработана в полном объёме и не размещена на официальном сайте университета, что может затруднить доступ и использование для широкого круга пользователей.

Рассмотрев три аналога можно сказать, что каждый из них имеет свои преимущества и недостатки.

Виртуальный тур по ОГУ предоставляет пользователю возможность ознакомиться с внутренним оформлением и интерьером различных зданий университета. Однако эта система не даёт полноценного понимания о планировке и внутренней структуре, поэтому никак не помогает при ориентировании внутри корпуса. Более того, виртуальный тур не предусматривает построение маршрутов, что особенно важно для новичков и гостей университета, нуждающихся в быстрых навигационных решениях.

Интерактивная 2D-карта МИРЭА представляет собой инструмент для ориентирования внутри зданий. Она позволяет выбирать корпуса, масштабировать и перемещать схему, переключаться между этажами и искать помещения по их номерам или названиям. В дополнение к этому, карта дает возможность строить маршруты между любыми точками внутри здания. Однако 2D-схема менее информативна с точки зрения визуального восприятия пространства: она не передает полноту трехмерного восприятия и не вызывает такого же уровня запоминания, как трехмерные модели [6].

Интерактивная карта-навигатор по корпусам И.С. Тургенева представляет собой простое, но эффективное решение для быстрого ориентирования внутри одного из двух корпусов ОГУ. Она позволяет просмотреть подробную схему и перемещаться по этажам, приближая или отдаляя изображение. Встроенная функция поиска помещений и построения маршрутов делает ее удобной для

пользователей, нуждающихся в быстром ориентировании. Однако система страдает от ограниченного числа корпусов — в настоящее время она охватывает только два здания, что существенно сужает её функциональные возможности, а также еще не полностью доработана и не размещена на официальном сайте университета, что мешает широкому использованию.

В целом, можно сказать, что каждая из систем обладает своими достоинствами. Виртуальный тур подходит для общего визуального ознакомления с интерьером зданий, что позволяет погрузиться в атмосферу университета. В то же время, интерактивная 2D-карта — это более точный инструмент, обеспечивающий навигацию внутри предоставленных корпусов. Объединение преимуществ этих решений и наглядной 3D модели создаст наиболее эффективный сервис ориентирования, который будет полезен для всех его пользователей.

Сравнение онлайн-сервиса с тремя другими аналогами представлено в таблице 1.2.1.

Таблица 1.2.1 – Анализ аналогов

Критерии сравнения \ Решения	Виртуальный тур ОГУ	МИРЭА	Интерактивная карта-навигатор по корпусам ОГУ им. И.С. Тургенева	Онлайн-сервис для ориентирования по корпусам
Возможность ознакомиться с внутренним оформлением университета	Да	Нет	Нет	Нет
Модель 3D	Да	Нет	Нет	Да
Наглядное представление планировки корпуса	Нет	Да	Да	Да
Поиск помещения	Нет	Да	Да	Да
Построение маршрута от начальной до конечной точки	Нет	Да	Да	Да
Масштабирование	Нет	Да	Да	Да
Наличие русского и английского языков	Нет	Нет	Нет	Да

Как видно из таблицы, реализация и внедрение этого решения повысит скорость адаптации абитуриентов и создаст положительное впечатление об университете, как о современном и удобном учебном пространстве.

1.3 Формулировка функциональных и нефункциональных требований

На основе анализа предметной области, изученных аналогичных решений и их функциональных возможностей, были выявлены основные функциональные и нефункциональные требования к сервису для ориентирования по корпусам университета.

Функциональные требования:

- возможность перемещения по моделям: переключение между корпусами, этажами;
- сброс вида модели (возврат к изначальному положению камеры);
- включение полноэкранного режима для страницы просмотра модели 3D корпуса;
- возможность опционального просмотра невыбранных этажей в полупрозрачном виде;
- возможность исключить построение маршрутов с выходом на улицу;
- возможность исключить построение маршрутов с лифтом;
- поддержка масштабирования, вращения и перемещения моделей для детального осмотра;
- отображение краткой информации о помещениях: номера кабинета или его назначения;
- вывод подсказок-инструкций по использованию графического интерфейса онлайн-сервиса;
- поиск выбранного помещения по его номеру или названию;
- расчёт и визуализация маршрутов внутри корпуса между двумя wybranными помещениями;
- возможность выбора русского или английского языка пользовательского интерфейса.

Нефункциональные требования:

- поддержка браузеров: использование версий браузеров Google Chrome, Яндекс.Браузер, Microsoft Edge или Mozilla Firefox с поддержкой технологии WebGL 1.0 (на базе версии Chromium 56 и выше для Windows и версии Chromium 66 и выше для Android);
- применение Mobile-first подхода к разработке и web-дизайну (дизайн и разработка начинаются с создания оптимальной версии для мобильных телефонов, только после этого, на основе мобильной, версии делается адаптация для более крупных экранов).

Требования к интерфейсу: дружелюбный, интуитивно понятный и эргономичный интерфейс.

При соблюдении всех функциональных и нефункциональных требований можно говорить о создании качественного онлайн-сервиса по ориентированию внутри корпусов университета. Такой сервис будет обладать удобным и интуитивно понятным интерфейсом, обеспечивая быстрый доступ к необходимой информации и маршрутам, а также поддерживая работу на различных устройствах и в популярных браузерах.

1.4 Обоснование и формулировка средств разработки онлайн-сервиса

Для реализации онлайн-сервиса был выбран единый язык программирования – JavaScript. Он применяется как на серверной, так и на клиентской стороне в браузере. Кроме того, для клиентской части выбрано два декларативных языка: HTML, формирующий структуру веб-страниц, и CSS, отвечающий за внешнее оформление и адаптивную верстку для различных размеров экранов.

Навигационная система состоит только из двух страниц с чётко разделённым функционалом. Пользовательский интерфейс содержит две страницы, одна из которых является страницей-каталогом для выбора корпуса, с динамически наполняемыми карточками, а вторая представляет из себя страницу просмотра модели с прямым манипулированием объектами сцены и двумя панелями управления. Функционала, предоставляемого JS, HTML и CSS, достаточно.

Для онлайн-сервиса была выбрана клиент-серверная архитектура с «тонким» сервером и «толстым» клиентом, поскольку серверная часть отвечает за небольшой функционал, в отличие от клиента, на которого ложатся все необходимые вычисления. Был выбран комбинированный подход к разработке – структурный на сервере, объектно-ориентированный на клиенте.

В качестве серверной платформы был выбран Node.js. Он позволяет единый язык программирования на сервере и клиенте, высокую производительность при обработке операций ввода-вывода и кроссплатформенность.

В качестве веб-фреймворка был выбран Express.js, поскольку он не диктует жёсткой структуры для разработчика, предоставляя свободу выбора архитектуры, упрощает обработку запросов (GET, POST и т.д.) и отправку ответов от сервера, а также имеет очень малое количество функций, что позволяет не перегружать проект лишними возможностями.

Такой стек соответствует серверной логике проекта. Сервер не нуждается в ООП подходе, поскольку решает одну линейную задачу. Он раздаёт статические файлы клиенту и отправляет плоский массив объектов в формате JSON при помощи API-метода, который читает содержимое директории, фильтрует его и формирует ответ.

На клиента ложится вся логика 3D-визуализации, построение графа навигации, поиск помещения и построение маршрута. Здесь применим подход ООП с присущими ему принципами – наследованием, инкапсуляцией, полиморфизмом.

Для визуализации 3D-моделей были использованы 3 библиотеки. Первая – `three.module.js` – предоставляет всё необходимое для создания и рендеринга трёхмерной сцены. Вторая – `OrbitControls.js`, она является официальным аддоном `Three.js`, реализующим стандартные жесты управления камерой. Третья – `GLTFLoader.js`, она также является официальным аддоном и отвечает за парсинг и загрузку 3D-моделей в формате glTF 2.0.

2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Разработка обобщённой архитектуры

Основная задача навигационной системы заключается в упрощении ориентирования по корпусу для всех его посетителей.

Функциональные требования предоставляют пользователям необходимые инструменты для быстрого и эффективного нахождения нужных помещений. Важным аспектом является возможность выбора языка интерфейса, что делает систему доступной для иностранных студентов и посетителей.

Нефункциональные требования акцентируются на технических аспектах, предоставляя пользователям соединение в браузерах, которые поддерживают технологии 1.0.

Для визуализации обобщённой архитектуры онлайн-сервиса была построена диаграмма развёртывания, представленная на рисунке 2.1.1. Она нужна для представления общей конфигурации и топологии системы и содержит распределение компонентов по отдельным её узлам. Кроме того, диаграмма показывает маршруты передачи информации между аппаратными устройствами. Всё это помогает понять взаимодействие компонентов и путей перемещения данных внутри системы.

Клиент и сервер взаимодействуют друг с другом посредством GET-запросов. Сервер принимает их, обрабатывает и формирует ответы, возвращаемые клиенту через сеть. Клиент отправляет запросы и получает ответы [7].

В данной архитектуре реализована модель толстый клиент. Сервер выполняет лишь задачу загрузки модели корпуса и передачи её клиенту, не участвуя в дальнейшем вычислительном процессе. В свою очередь, клиент самостоятельно проводит все необходимые вычисления алгоритмов для построения маршрутов и взаимодействия с моделью, обеспечивая более эффективное распределение нагрузки.

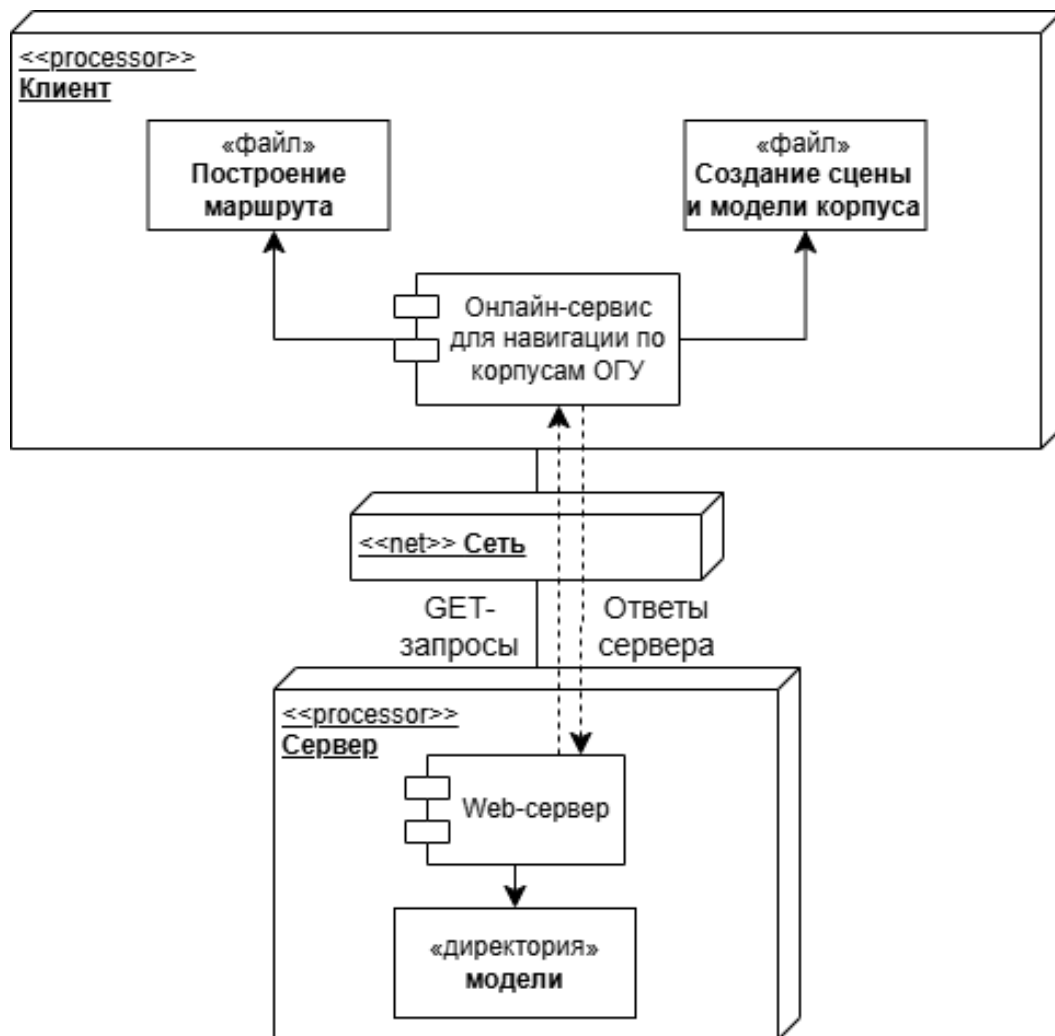


Рисунок 2.1.1 – Архитектура онлайн-сервиса

Такой подход позволяет значительно снизить нагрузку на сервер и обеспечить более быстрый отклик пользовательского интерфейса.

При этом вопрос со сложным обеспечением безопасности данных в данном случае не является большой проблемой, поскольку в системе отсутствуют механизмы авторизации или строгой защиты информации. Это обусловлено тем, что все пользователи имеют равные права и доступ к функционалу сервиса.

2.2 Разработка функциональных спецификаций

Для определения возможностей, доступных пользователю, наиболее эффективным является создание диаграммы вариантов использования [8]. Она позволяет описать взаимодействие актёров (студентов, преподавателей, работников и т.д.) с вариантами использования (функциями системы), не

углубляясь в детали реализации. Диаграмма, представленная на рисунке 2.2.1, отображает функциональные требования с точки зрения пользователя, показывая, какой функционал ему доступен.

Для начала необходимо определить актёров. Поскольку онлайн-сервис предоставляет одинаковые функциональные возможности для всех категорий пользователей, будь то гости, преподаватели, студенты или сотрудники ВУЗа, то все они объединены в одного актёра с названием «Пользователь».

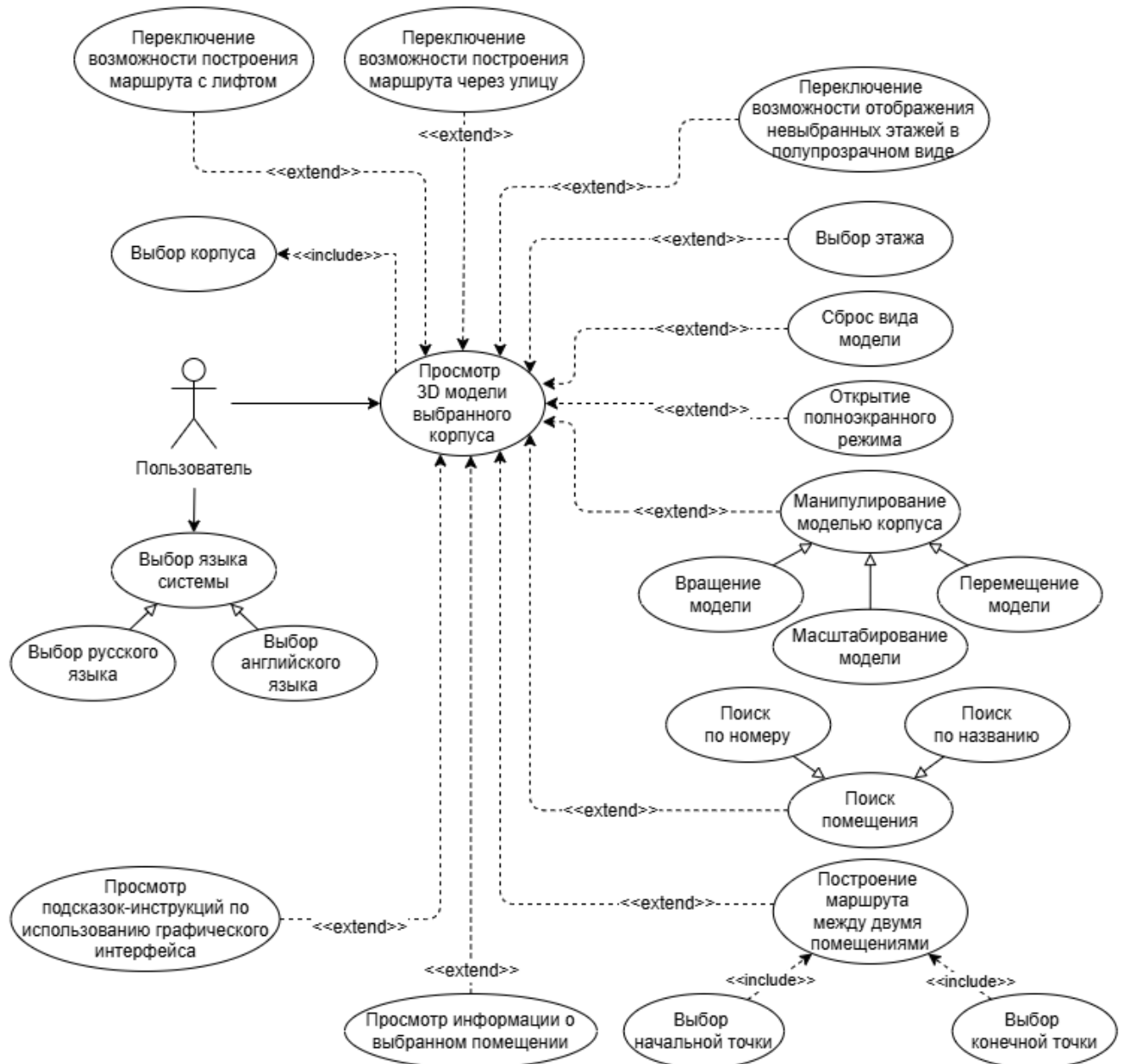


Рисунок 2.2.1 – Функциональная спецификация онлайн-сервиса

Основные действия пользователя – просмотр 3D-модели выбранного корпуса и выбор языка системы.

Пользователь может просматривать 3D-модель только после выбора конкретного корпуса, для которого ему нужна навигация.

При просмотре модели становятся доступными и другие функции, такие как манипулирование моделью (вращение, перемещение и масштабирование), сброс вида модели, открытие полноэкранного режима, отображение подсказок-инструкций по использованию графического интерфейса. Кроме того, он может выбирать этаж, который будет отображаться на модели, при этом можно скрыть остальные этажи или показывать их полупрозрачными. Пользователь может получить информацию о любом помещении, расположенном на модели двумя способами: дважды нажав на дверь, отвечающую за кабинет, или найдя его по номеру или названию. Так же он может строить маршрут между двумя помещениями, указывая начальную и конечную точки. При этом, пользователь может скрыть маршруты, проходящие через лифт или через улицу.

2.3 Разработка информационных спецификаций

Наиболее наглядным и понятным способом определения структуры данных является создание логической диаграммы классов. Она хорошо описывает неоднородные объекты с данными, совместно участвующие в функционировании онлайн-сервиса.

Диаграмма классов представлена на рисунке 2.3.1.

Диаграмма содержит 8 основных классов: корпус, этаж, область, точка, помещение, главная и вспомогательная точки, а также навигационный граф. Корпус включает в себя объекты класса «Этаж». Этаж включает в себя объекты класса «Область». Область включает в себя объекты класса «Точка».

Класс «Точка» является родительским классом для помещения, главной и вспомогательной точек.

Такая структура позволяет подробно и точно описать учебные корпуса для дальнейшей навигации в них. Связи между корпусом, этажом, областью и точкой отражают физическую организацию пространства в здании, где каждый новый уровень детализации уточняет расположение объекта, а наличие потомков у класса

точки позволяет различать функциональное назначение точек навигации. Главные точки служат для перехода между областями, вспомогательные точки нужны для более точного проведения маршрута в области, точки помещения представляют из себя аудитории с номером и описанием.

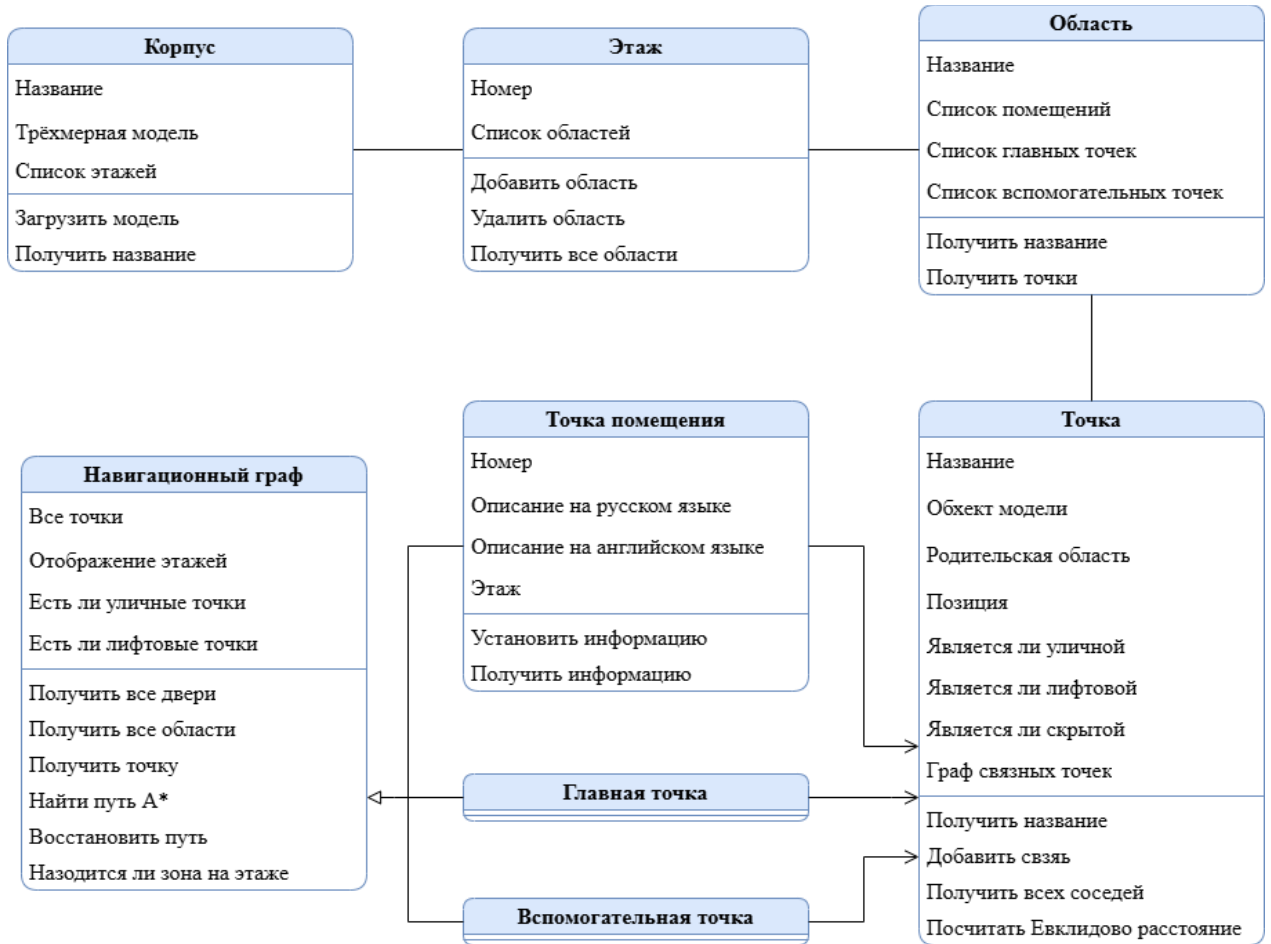


Рисунок 2.3.1 – Информационная спецификация

Класс «Навигационный граф» является классом, который ответственен за централизованное хранение всех навигационных точек и связей между ними. Кроме того, именно в этом классе будет реализована функция поиска маршрута между двумя помещениями.

2.4 Разработка поведенческих спецификаций

Поведенческие спецификации позволяют понять, как система будет вести себя в различных сценариях использования, обеспечивая правильную работу и взаимодействие с пользователем.

Одним из способов моделирования поведения является диаграмма последовательности, представленная на рисунке 2.4.1. Она иллюстрирует действия, производимые пользователем, с момента открытия онлайн-сервиса и до момента построения маршрута между двумя точками. Это является ключевым функционалом онлайн-сервиса.

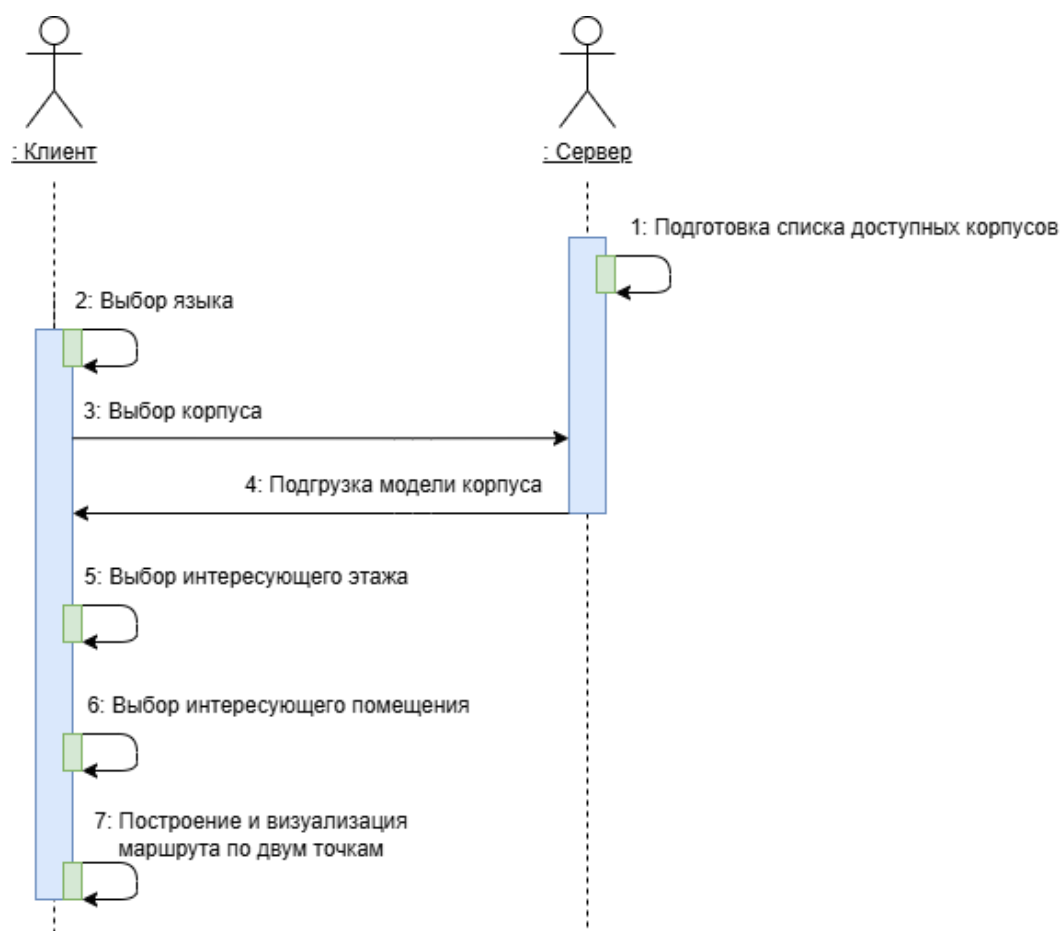


Рисунок 2.4.1 – Взаимодействие компонентов системы при построении маршрута

На данной диаграмме объектами, участвующими во взаимодействии, являются пользователь и онлайн-сервис. Анализируя диаграмму можно заметить, что линия жизни онлайн-сервиса – временной промежуток, в течение которого объект существует в системе и способен участвовать во всех её взаимодействиях – не проходит по всей плоскости диаграммы последовательности. Это означает, что объект участвует не во всех этапах сценария и что отсутствует необходимость в его постоянном взаимодействии с пользователем. В обязанности онлайн-сервиса входит выполнение задач, связанных с подготовкой и предоставлением списка доступных зданий, а также загрузка модели и вспомогательных файлов для

выбранного корпуса. Все остальные задачи, такие как выбор языка, этажа или помещения, а также алгоритм поиска пути, его построение и визуализация осуществляются пользователем.

Исходя из этого можно сделать вывод, что онлайн-сервис выполняет функции вспомогательного характера, в то время как пользователь управляет основными. Это снижает нагрузку на сервис и уменьшает задержки, связанные с постоянной передачей данных, что обеспечивает более быструю и гибкую его работу. В условиях большого числа одновременных пользователей, данный подход позволяет обеспечить стабильную работу сервиса без существенных ухудшений в его производительности.

Для полного понимания поведения системы, необходимо создать диаграмму состояний и переходов [9]. Она помогает понять то, как работает система в различных ситуациях, как она реагирует на внешние воздействия, какие действия выполняет в каждом состоянии и как происходит переход к следующему этапу.

Состояние – это описание конкретного состояния системы, которое может изменяться в зависимости от её взаимодействия со внешней средой. Переход – это описание процесса изменения системы, он описывает условия, которые должны быть выполнены, чтобы система перешла из одного состояния в другое.

Диаграмма состояний и переходов представлена на рисунке 2.4.2.

Навигационная система для ориентирования по корпусам университета является онлайн-сервисом, именно поэтому пользователь имеет возможность выйти из неё из любого её состояния.

Начальным состоянием является просмотр списка доступных корпусов. Там пользователь может выбрать язык системы и интересующую его 3D-модель из предложенных вариантов. После этого происходит переход к состоянию просмотра 3D-модели выбранного корпуса.

В состоянии просмотра 3D-модели можно взаимодействовать со сценой, на которой располагается модель: масштабировать, вращать, перемещаться, сбрасывать вид до изначального. Кроме того, можно перейти в полноэкранный режим для большего удобства.

Верхняя панель имеет несколько важных функций. Она позволяет вернуться к просмотру списка доступных корпусов, переключить язык или отобразить/скрыть подсказку, объясняющую как пользоваться навигационной системой.

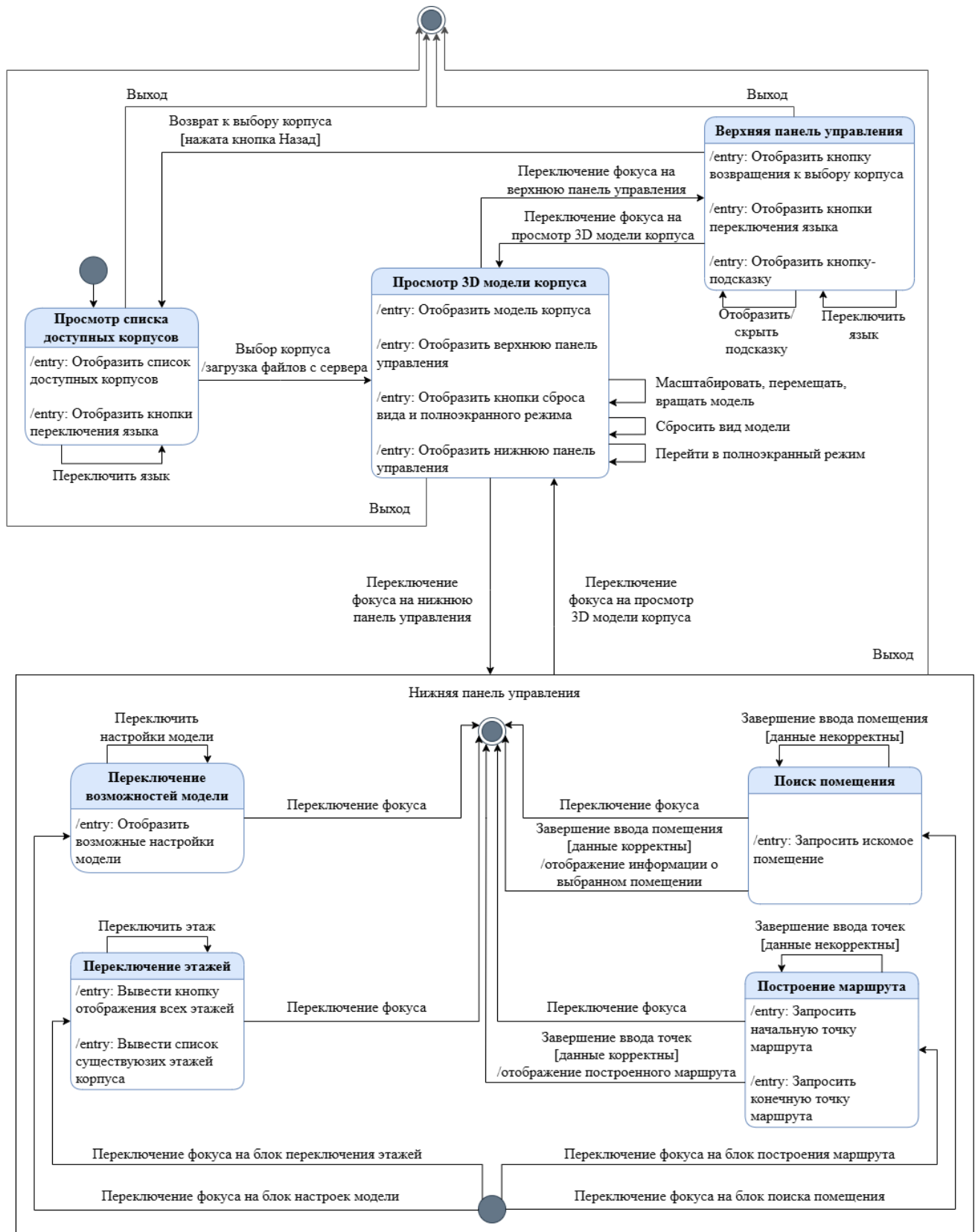


Рисунок 2.4.2 – Поведенческая спецификация

Нижняя панель управления разбита на 4 секции, каждая из которых отвечает за свои функции.

Раздел переключения возможностей модели позволяет изменить такие настройки, как прозрачность невыбранных этажей или особенности, которые будут учитываться при построении маршрута. Количество этих переключателей зависит от корпуса, например, если корпус содержит лифт, то появится переключатель, позволяющий исключить лифт из построения маршрута.

Раздел переключения этажей позволяет пользователю выбрать активный этаж, а остальные отобразить в полупрозрачном виде или не отобразить вовсе, в зависимости от настроек модели. Кроме того, есть возможность выбрать всё этажи, отобразив их в полностью непрозрачном виде.

Раздел поиска помещения содержит строку ввода, которая, для удобства пользователя, дополнена поиском по кабинетам. Для нахождения необходимо выбрать искомое помещение из выпадающего списка и нажать кнопку поиска. Если данные были корректны, то ниже этого раздела появится найденная информация, такая как этаж, номер и название. Кроме того, камера переместится к найденному помещению, дверь которого будет подсвечиваться.

Раздел построения маршрута предоставляет возможность построить маршрут по двум точкам. Для этого необходимо заполнить оба поля ввода, выбрав помещения из выпадающего списка, который, также со встроенным поиском. Если было заполнено только одно поле, то появляется предупреждение о том, что необходимо заполнить оба. При правильном заполнении и нажатии кнопки поиска происходит отображение построенного маршрута и одновременное перемещение к его начальной точке.

Так же необходимо отметить, что после выбора корпуса и перехода к состоянию просмотра 3D-модели, пользователь может свободно перемещаться между любыми элементами интерфейса (верхняя и нижняя панели управления, а также состояние просмотра корпуса) без необходимости каждый раз возвращаться к состоянию просмотра. Однако эти связи решено было не добавлять в диаграмму, чтобы не нагружать её внешний вид.

3 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1 Проектирование структуры

На рисунке 3.1.1 изображена диаграмма структуры онлайн-сервиса, на которой представлены основные модули и связи между ними [10].

В серверной части основным компонентом является файл `server.js`, который содержит основной серверный скрипт на Node.js. Он отвечает за обработку GET-запросов от пользователей, передачу статических файлов и за интерфейс, возвращающий клиенту информацию обо всех доступных моделях корпусов.

Так же в серверной части находится директория `models`, которая хранит в поддиректории `models/building_N`, где находятся файлы с моделью для каждого корпуса, а также вспомогательные файлы: `AreaN`, отвечающий за группировку точек по областям; `Floor_areas`, группирующий области по этажам; `Doors_metadata`, наполняющий помещения дополнительной информацией (номер, описание) и `Points_tags`, необходимый для обозначения особых точек.

Клиентская часть содержит множество файлов и директорий, поскольку все вычисления выполняются на стороне пользователя, а не сервера.

Директория `public` содержит компоненты `index.html` и `viewer.html`. Они являются страницами, на которых отображается список всех доступных корпусов и отображение конкретного корпуса соответственно.

Кроме того, `public` содержит директорию `css`, в которой находятся файлы со стилями (`index.css` и `viewer.css`), они создают визуальное оформление веб-страниц. В директории `assets`, содержащей папку `images`, хранятся изображения корпусов, которые необходимы для отображения на странице выбора корпуса.

Директория `public` так же содержит в себе папку `js`, в которой находятся файлы, реализующие логику онлайн-сервиса.

Файл `index.js` отвечает за управление главной страницей, а именно за загрузку списка корпусов, формирование карточек и переход к просмотру корпуса.

Файл `viewer.js` загружает модели, строит навигационный граф, отвечает за настройку обработчиков событий.

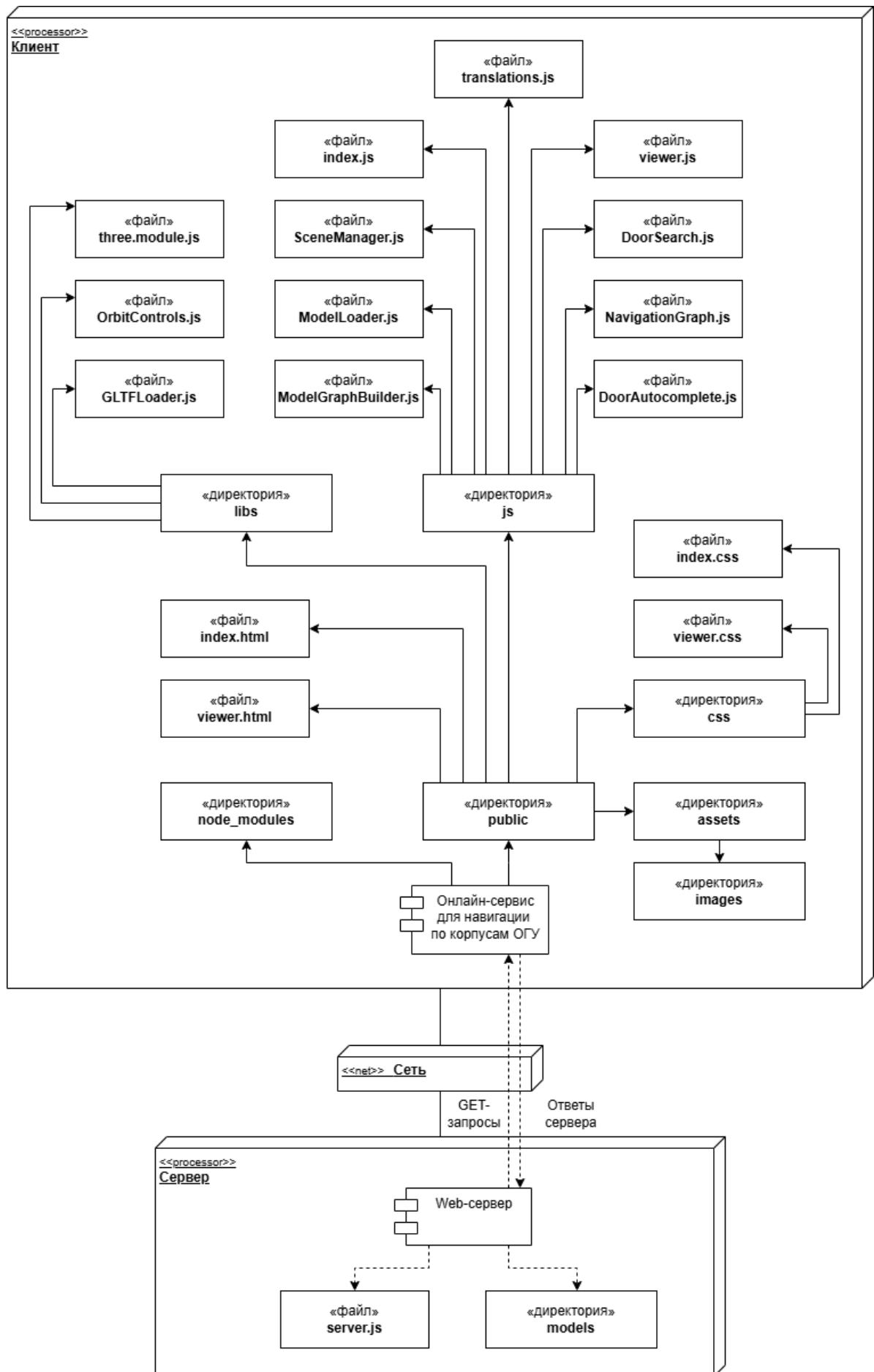


Рисунок 3.1.1 – Структура онлайн-сервиса

Файл translation.js поддерживает локализацию, храня информацию на русском и английском языках и обновляя тексты на страницах.

Файл DoorAuthocomplete.js работает поиском, отображая выпадающий список в отсортированном виде, так же ответственен за обработку выбора.

Файл DoorSearch.js управляет поиском помещения и связанными с ним действиями, такими как отображение информации, подсветка двери, переключение этажа и наведение камеры.

Файл ModelLoader.js загружает glb-файлы моделей корпуса с сервера и осуществляет их кэширование для предотвращения повторного скачивания при возврате в меню выбора корпуса и обратно.

Файл ModelGraphBuilder.js извлекает из 3D-модели навигационные точки, загружает все вспомогательные файлы и строит граф навигации.

Файл NavigationGraph.js хранит все точки и связи и реализует алгоритм A* для поиска кратчайшего маршрута между двумя точками.

Файл SceneManager.js управляет сценой, камерой, освещением, отвечает за отображение и скрытие этажей, их прозрачность, кроме того, этот файл отвечает за графическую визуализацию маршрута.

3.2 Проектирование компонентов

Для автоматизированного извлечения объектов из glb файла и правильной работы навигационной системы необходимо, чтобы каждая модель корпуса имела чётко определённую внутреннюю иерархию коллекций. Структура модели в Blender представлена на рисунке 3.2.1.

Blender содержит корневую коллекцию SceneCollection, которая содержит в себе все остальные. Это обязательный системный уровень, который нельзя удалить, пока существует сцена [11].

Затем идёт созданная коллекция для модели корпуса Building, которая имеет две дочерние коллекции: Floorn и WayPointsn.

Floorn – это коллекция этажа, она содержит в себе все объекты, которые необходимы для его визуального представления: стены, полы, декоративные

элементы. Экземпляров этой коллекции может быть много.

WayPoints n – это коллекция, отвечающая за навигационную структуру. Внутри неё находятся коллекции Area n .

Каждая Area n соответствует определённой области навигации и содержит в себе 3 специализированные коллекции: Doors n , Primary n , Secondary n .

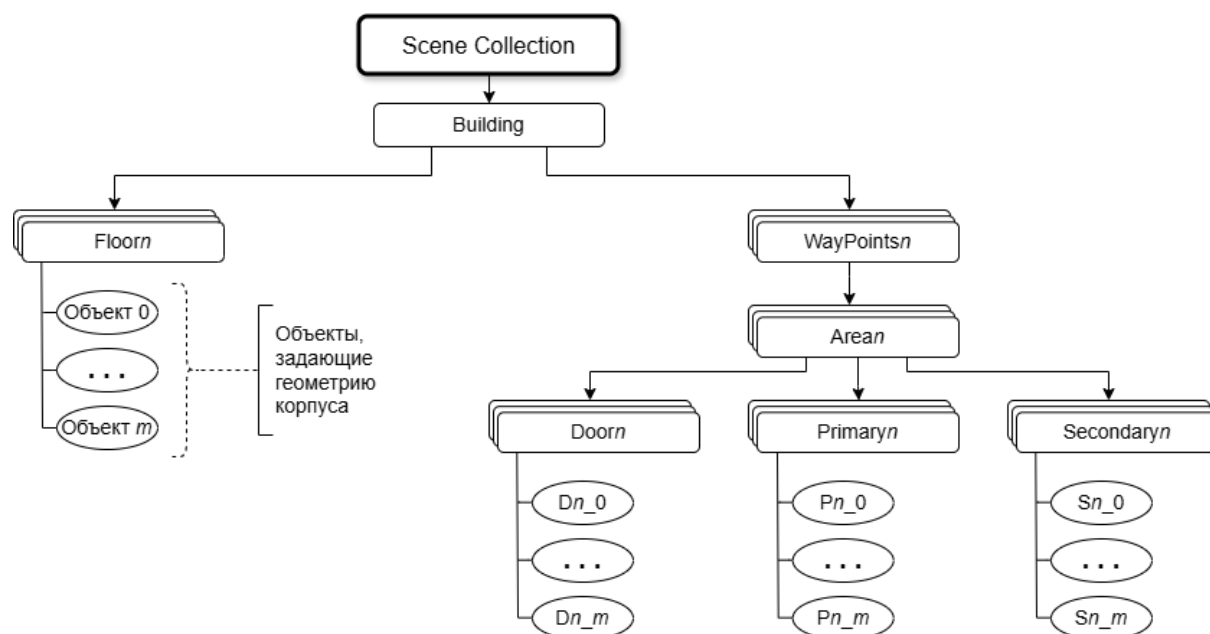


Рисунок 3.2.1 – Структура модели в Blender

Коллекция Doors n содержит объекты с именами формата D n _ n , где первая буква D обозначает тип объекта (дверь), первое число обозначает номер Area (области), а второе число является порядковым номером двери. Объекты этой коллекции представляют собой искомые пользователем точки, двери, то есть помещения, которые им необходимо найти или между которыми необходимо проложить маршрут.

Коллекция Primary n содержит объекты с именами формата P n _ n , где первая буква P обозначает тип объекта (главная точка), первое число обозначает номер Area (области), а второе число является порядковым номером точки. Объекты этой коллекции служат связующими между различными областями (Area), поскольку только через эти точки возможен переход из одной области в другую.

Коллекция Secondary n содержит объекты с именами формата S n _ n , где первая буква S обозначает тип объекта (вспомогательная точка), первое число обозначает номер области (Area), а второе число является порядковым номером

точки. Объекты этой коллекции обеспечивают детальную навигацию внутри одной области (Area), связывая Primary точки между собой и позволяя строить маршруты, огибая такие особенности области, как колонны, стены и т.д.

Такая структура модели в Blender позволяет классам извлекать данные для взаимодействия с моделью, построения маршрута, поиска помещения и других функций. Описание классов, их полей, методов и назначения представлено в таблице 3.2.1.

Таблица 3.2.1 – Описание классов, их полей, методов и назначения

Класс	Поля класса	Методы класса
ModelGraphBuilder	graph buildingId	build(model, buildingId) extractWaypoints(model) findWaypointsCollection(model) createAreaFromObject(areaObject) hideNonDoors() loadFloorMapping() loadConnections() collectAreaIndices() parseConnectionsFile(content) loadDoorInfo() loadPointsTags()
ModelLoader	cache loadingPromises GLTFLoader initPromise	initLoader() loadModel(buildingId) fetchModel(buildingId)
Point	id modelObject parentArea connections position isStreet isElevator isHidden	addConnection(targetPoint) getAllConnections() getEuclideanDistance(targetPoint)

Продолжение таблицы 3.2.1

Класс	Поля класса	Методы класса
PrimaryPoint (наследник Point)	type	
SecondaryPoint (наследник Point)	type	
DoorPoint (наследник Point)	type number descriptionRu descriptionEn floor	setInfo(number, descriptionRu, descriptionEn, floorMapping) getDescription(lang) getInfo(lang)
Area	index	extractPoints(areaObject) getPointClass(collectionName) createPoint(pointObject, PointClass)
NavigationGraph	allPoints floorMapping hasStreetPoints hasElevatorPoints	navigateAStar(startDoorID, endDoorID, options) reconstructPath(path, current) getPoint(id) getAllPoints() getAllDoors() getAreasForFloor(floorNumber) isAreaOnFloor(areaIndex, floorNumber)
DoorSearch	input dropdown doorInfo autocomplete highlightedDoor	showDoor(door, focus) showInfo(door) updateLanguage() hideInfo() highlightDoor(door) clearDoorHighlight() startDoorPulseAnimation(door) moveToFloor(door) clear() findByQuery(query) handleSearch()

Продолжение таблицы 3.2.1

Класс	Поля класса	Методы класса
SceneManager	canvas scene camera renderer controls model animationId floors currentFloor floorMaterials transparentMode	init() setupLights() setModel(glb) analyzeFloors() findFloorsInCollection(collection) getFloorList() showAllFloors() setTransparentMode(enabled) getTransparentMode() showFloor(floorNumber) setObjectOpacity(object, opacity) resetView() render() resize(width, height) dispose() focusOnPoint(point, distance, verticalOffset) visualizePath(path) clearPath()
DoorAutocomplete	input dropdown allDoors filteredDoors	init() getDisplayText(door) refresh() reloadDoors() onInput() selectDoor(door) show() / hide() resetFilter() render()

Каждый из вышеописанных классов имеет своё назначение.

Класс «ModelGraphBuilder» загружает 3D модель корпуса, парсит файлы связей и метаданные для дверей, строит граф навигации.

Класс «ModelLoader» загружает glb-модели корпусов и защищает от повторной загрузки при открытии модели здания.

Класс «Point» является базовым классом для всех точек графа.

Класс «PrimaryPoint» – это главная точка, которые соединяет две соседние области друг с другом.

Класс «SecondaryPoint» – это второстепенная точка, которая помогает прокладывать маршрут внутри области.

Класс «DoorPoint» – это точка двери, которая является только начальной и конечной точками. Через эту точку нельзя прокладывать маршруты.

Класс «Area» извлекает все точки из соответствующей области в 3D-модели.

Класс «NavigationGraph» хранит полный граф навигации и реализует алгоритм A*, ищущий кратчайший путь с учётом настроек модели.

Класс «DoorAutocomplete» отвечает за выпадающий список для всех полей ввода.

Класс «SceneManager» управляет 3D-сценой.

Класс «DoorSearch» обеспечивает поиск и отображение информации о двери, а также отвечает за графическое выделение найденной двери.

3.3 Проектирование структур данных

Были спроектированы следующие физические структуры для хранения входных, промежуточных и выходных данных.

Входные файлы навигационной системы являются источниками данных. Каждый файл решает свою отдельную задачу.

GLB-файлы для модели. Каждый корпус представлен одним бинарным файлом building_N.glb. Такой формат файла объединяет сцены, узлы, меши, материалы и текстуры, даёт доступ к буферам геометрии и позволяет извлечь иерархию объектов с пользовательскими именами. Формат glb был выбран, поскольку имеет ряд преимуществ:

- небольшой размер – бинарный формат занимает меньше места, чем текстовый gltf;

- единый файл – легче загрузить, кэшировать, не требуется брать текстуры, сцены и геометрии из разных файлов;
- поддержка Three.js – работает стандартный загрузчик GLTFLoader;
- пользовательские настройки – иерархия и имена объектов остаются такими, какими создал и назвал их пользователь, что крайне важно для корректного извлечения данных.

Без этого файла невозможно отобразить 3D-модель корпуса, определить координаты точек и визуализировать маршрут.

Файл AreaN.txt описывает связи между точками, находящимися в одной области. На рисунке 3.3.1 изображено графическое представление файла Area7.txt, а на рисунке 3.3.2 – реальное содержание этого же файла.

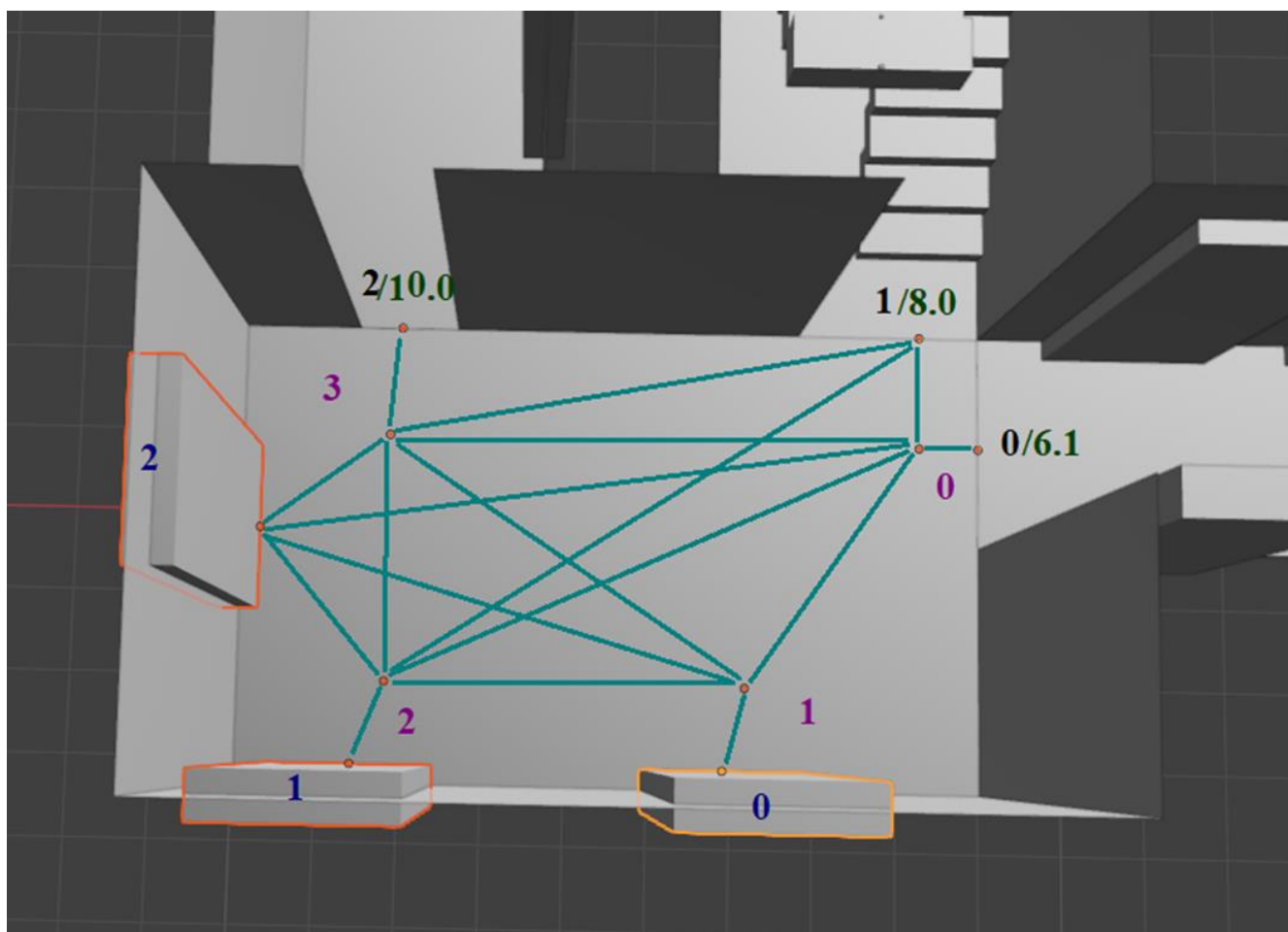


Рисунок 3.3.1 – Графическое представление файла Area7

Синим цветом отмечена нумерация дверей, чёрным цветом – нумерация главных точек, фиолетовым – нумерация второстепенных точек, тёмно-зелёным – главные точки смежных областей, к которым можно перейти из текущей.

```

P7_0: S7_0, P6_1
P7_1: S7_0, S7_2, S7_3, P8_0
P7_2: S7_3, P10_0
S7_0: S7_1, S7_2, S7_3, D7_2, P7_0, P7_1
S7_1: S7_0, S7_2, S7_3, D7_0, D7_2
S7_2: S7_0, S7_1, S7_3, D7_1, S7_2, P7_1
S7_3: S7_0, S7_1, S7_2, D7_2, P7_1, P7_2
D7_0: S7_1
D7_1: S7_2
D7_2: S7_0, S7_1, S7_2, S7_3

```

Рисунок 3.3.2 – Файл Area7

Главные точки обозначаются как Pn_n, второстепенные точки как Sn_n, а точки дверей как Dn_n.

Переходить в соседние области возможно только по главным точкам. Например, чтобы перейти из главной точки P7_0 в главную точку P8_0, необходимо проделать следующий маршрут: P7_0 → S7_0 → P7_1 → P8_0.

Файлы AreaN необходимы, поскольку не обязательно соседние точки будут соединены друг с другом, так как между ними может оказаться стена.

Файл Floor_areas.txt устанавливает соответствия между этажами корпуса и принадлежащими к ним областями. Эта информация необходима при переключении этажа (чтобы знать какие области необходимо скрыть или показать), при определении этажа для конкретной двери и при фильтрации точек при поиске. Пример файла представлен на рисунке 3.3.3.

```

-1: 0, 1, 2, 3
1: 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26

```

Рисунок 3.3.3 – Файл Floor_areas

Файл Doors_metadata.json хранит метаданные для каждой двери: номер, если есть и описание на русском и английском языках, если есть. Эта информация отображается в интерфейсе пользователя, когда ему выводится информация о найденном помещении.

Без этого файла пользователь мог бы получать только идентификаторы двери, что не информативно и противоречит безопасности. Пример части файла представлен на рисунке 3.3.4.

```
{
  "D1_0": {
    "description_ru": "Гардероб",
    "description_en": "Wardrobe"
  },
  "D6_1": {
    "number": "101",
    "description_ru": "Преподавательская",
    "description_en": "Teaching staff"
  },
  "D12_4": {
    "number": "125"
  },
  "D12_5": {
    "number": "126"
  },
}
```

Рисунок 3.3.4 – Файл Doors_metadata

Как видно из изображения, дверь, она же помещение, может не иметь описания или номера. Это является нормой, в таком случае эти сведения просто не будут отображаться пользователю.

Файл Points_tags.txt может назначать точкам специальные флаги, из-за которых они могут не участвовать в построении маршрута, поскольку пользователь может запретить построение маршрута через улицу или лифт. Пример файла представлен на рисунке 3.3.5.

```
street: P26_1, P26_2, P26_3, P26_4, S26_1, S26_2, S26_3, S26_4, S26_5, S26_6, S26_7
elevator:
hidden: D12_3, D15_1
```

Рисунок 3.3.5 – Файл Points_tags

Street отвечает за точки, которые будут вести через улицу, elevator за точки, которые будут вести через лифт, а hidden за те точки, которые должны будут убираться из маршрута, поскольку они являются вспомогательными.

Как видно из рисунка, после двоеточия может не идти ни одной точки, тогда это означает, что в данном корпусе ни одна точка не соответствует этому условию.

Все эти файлы находятся в директории building_n и вместе дают полную

информацию о модели конкретного корпуса. Это даст возможность использовать весь существующий функционал для навигации по зданию.

Преимущества такого разделения по файлам заключается в том, что можно независимо обновлять различные данные, не трогая модель и код программы, кроме того текстовые файлы легко редактировать, тестировать и править, что крайне важно при работе с масштабируемыми проектами.

3.4 Проектирование алгоритмов

Онлайн-сервис для ориентирования по корпусам имеет множество алгоритмов, но основным из них является алгоритм построения маршрута. Для реализации был выбран алгоритм A^* , как один из самых эффективных для поиска кратчайшего пути между двумя точками.

Были рассмотрены два классических алгоритма – Дейкстра и A^* .

Алгоритм Дейкстры находит кратчайший путь от начальной до конечной точки, что является его преимуществом. Но метод направлен во все стороны, он не использует информацию о целевой точке, он находит пути до всех вершин, что, в данном случае, является его недостатком.

Алгоритм A^* напротив, должен знать координаты конечной точки, поскольку он использует эвристическую функцию для направленного поиска. Для каждого узла он вычисляет следующую формулу

$$f(n) = g(n) + h(n), \quad (1)$$

где $f(n)$ – общая стоимость пути;

$g(n)$ – точная стоимость пути от начальной вершины до текущей;

$h(n)$ – эвристическая оценка расстояния от текущей вершины до конечной.

В качестве эвристической оценки будет использоваться евклидово расстояние между трёхмерными координатами точек, что позволит A^* двигаться по тем точкам, которые находятся в нужном направлении, в стороне конечной точки.

В таблице 3.3.1 наглядно представлены основные различия между алгоритмами Дейкстры и A^* .

Таблица 3.3.1 – Различия алгоритмов Дейкстра и A*

Алгоритм Характеристика	Дейкстра	A*
Использование информации о целевой точке	Не использует	Использует
Направленность	Расширение во все стороны (радар)	Направленность в сторону целевой точки
Скорость работы	Значительно падает на больших графах	Гораздо быстрее за счёт отсутствия лишних расчётов
Оптимальность	Всегда находит кратчайший путь	Находит кратчайший путь при не переоценённом реальном расстоянии
Требуемая память	Больше, т.к. хранит все точки при поиске во все стороны	Меньше, т.к. хранит все вершины, но обычно ищет сразу в правильном направлении
Применимость	Работает на графах без дополнительной информации	Требуется координаты точек для оценки расстояния до цели

Для разработки онлайн-сервиса навигации по корпусам ОГУ лучше всего подходит именно алгоритм A* сразу по нескольким причинам.

Во-первых, поскольку легко получить координаты начальной и конечной точек, то необходимо их использовать, если это может помочь ускорить алгоритм.

Во-вторых, в качестве эвристической функции будет использоваться Евклидово расстояние, которое использует координаты трёхмерных точек. Это позволит не переоценить реальное расстояние между точками.

В-третьих, граф навигации корпуса может быть достаточно большим, поскольку в среднем имеет несколько сотен точек. В таком случае алгоритм Дейкстры мог бы приводить к заметным задержкам на слабых устройствах, что

показано в таблице 3.3.2 и на рисунке 3.3.1.

Таблица 3.3.2 – Экспериментальное сравнение алгоритмов Дейкстра и A*

Категория расстояния	Среднее количество обработанных вершин (Дейкстра)	Среднее количество обработанных вершин (A*)	Коэффициент ускорения (раз)
Короткое (≤ 20 м)	78	18	4,3
Среднее (20–50 м)	215	52	4,1
Длинное (> 50 м)	437	112	3,9
В среднем	243	61	4

Для сравнения был создан граф, содержащий 500 вершин, средняя степень который равна 3. Затем случайным образом было выбрано 100 пар вершин. Результаты усреднялись по 100 запросам отдельно для каждой группы.

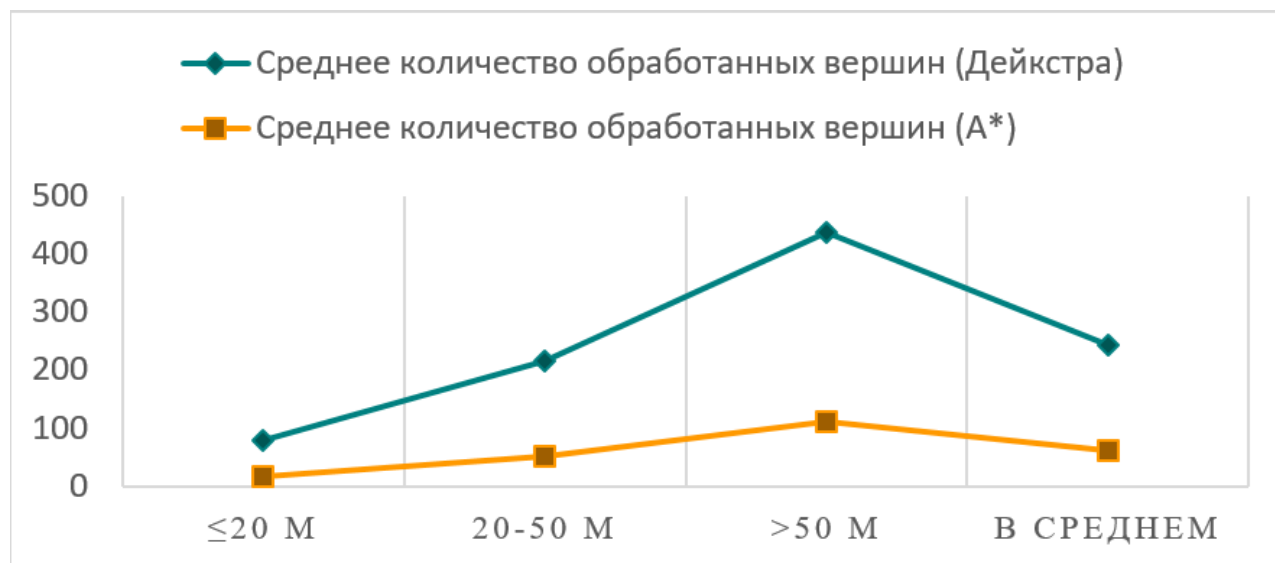


Рисунок 3.3.1 – График сравнения алгоритмов Дейкстра и A*

Таким образом, выбор алгоритма A* является наилучшим для текущей задачи.

Подробное описание алгоритма A* представлено на рисунке 3.3.2.

Логика поиска маршрута разбивается на две основные ветви в зависимости от того, совпадают ли начальная и конечная точки. Если это так, то вызывается функция поиска помещения, иначе отрабатывает проиллюстрированный алгоритм.

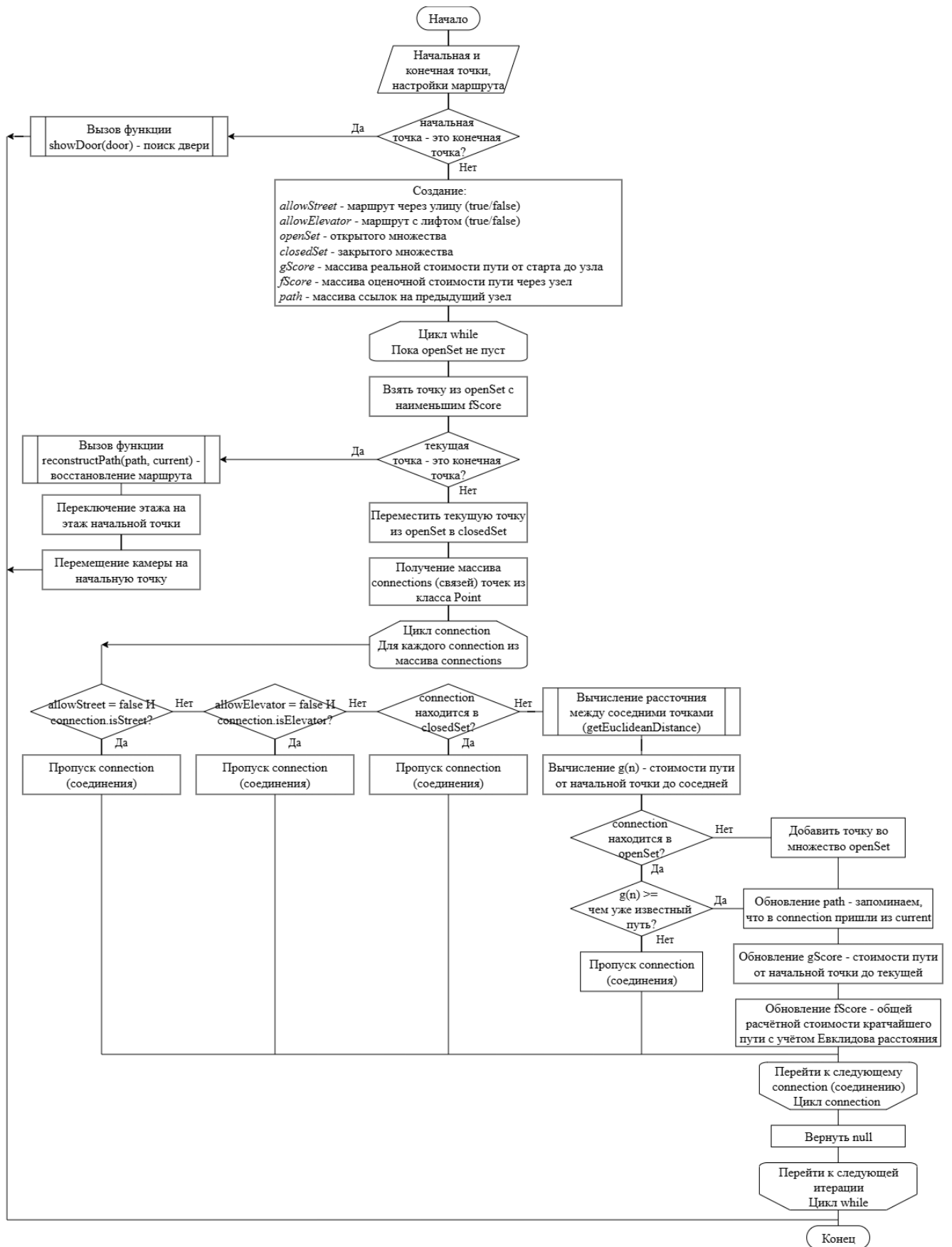


Рисунок 3.3.2 – Схема алгоритма поиска маршрута между двумя точками

В цикле «while» происходит основной перебор точек из открытого

множества. Если эта точка является искомой, то вызывается функция восстановления пути. После её отработки происходит переключение этажа на этаж начальной точки, а также происходит перемещение камеры таким образом, чтобы начальная точка оказалась в её центре.

В противном случае точка перемещается в закрытое множества и происходит обработка всех её соседей. Если связь удовлетворяет условиям, то происходит вычисление евклидова расстояния между соседними точками. Затем, если связь не находится в открытом множестве, то она туда добавляется. Иначе сравнивается новая полученная стоимость маршрута от начала до этой точки с той стоимостью, что сохранена в открытом множестве. Если новая стоимость меньше, то происходит обновление параметров и переход к следующему соседу.

Если искомая точка так и не была найдена, то цикл завершается и пользователю выводится сообщение о том, что путь не найден.

Этот алгоритм является основным, но содержит в себе и другие важные функции, которые будут рассмотрены далее.

На рисунке 3.3.3 изображена эвристическая функция, вычисляющая евклидово расстояние в трёхмерном пространстве между текущей и целевой точкой.

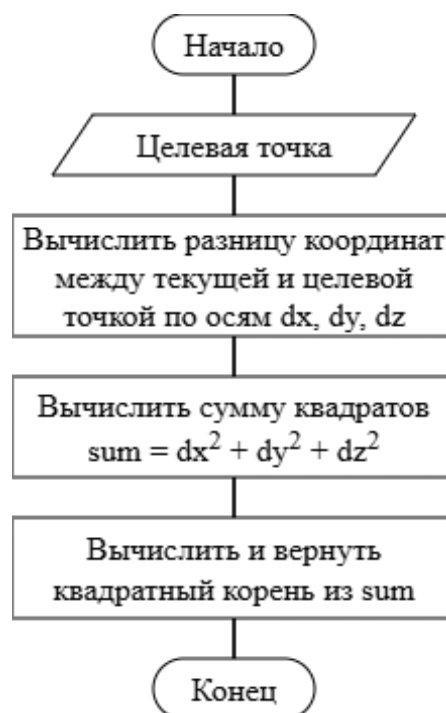


Рисунок 3.3.3 – Схема алгоритма вычисления евклидова расстояния

Алгоритм вычисляет разности координат по каждой оси (dx , dy , dz), а затем суммирует квадраты этих разностей. Из полученной суммы извлекается квадратный корень, который и является евклидовым расстоянием между точками.

На рисунке 3.3.4 изображена функция восстановления маршрута.

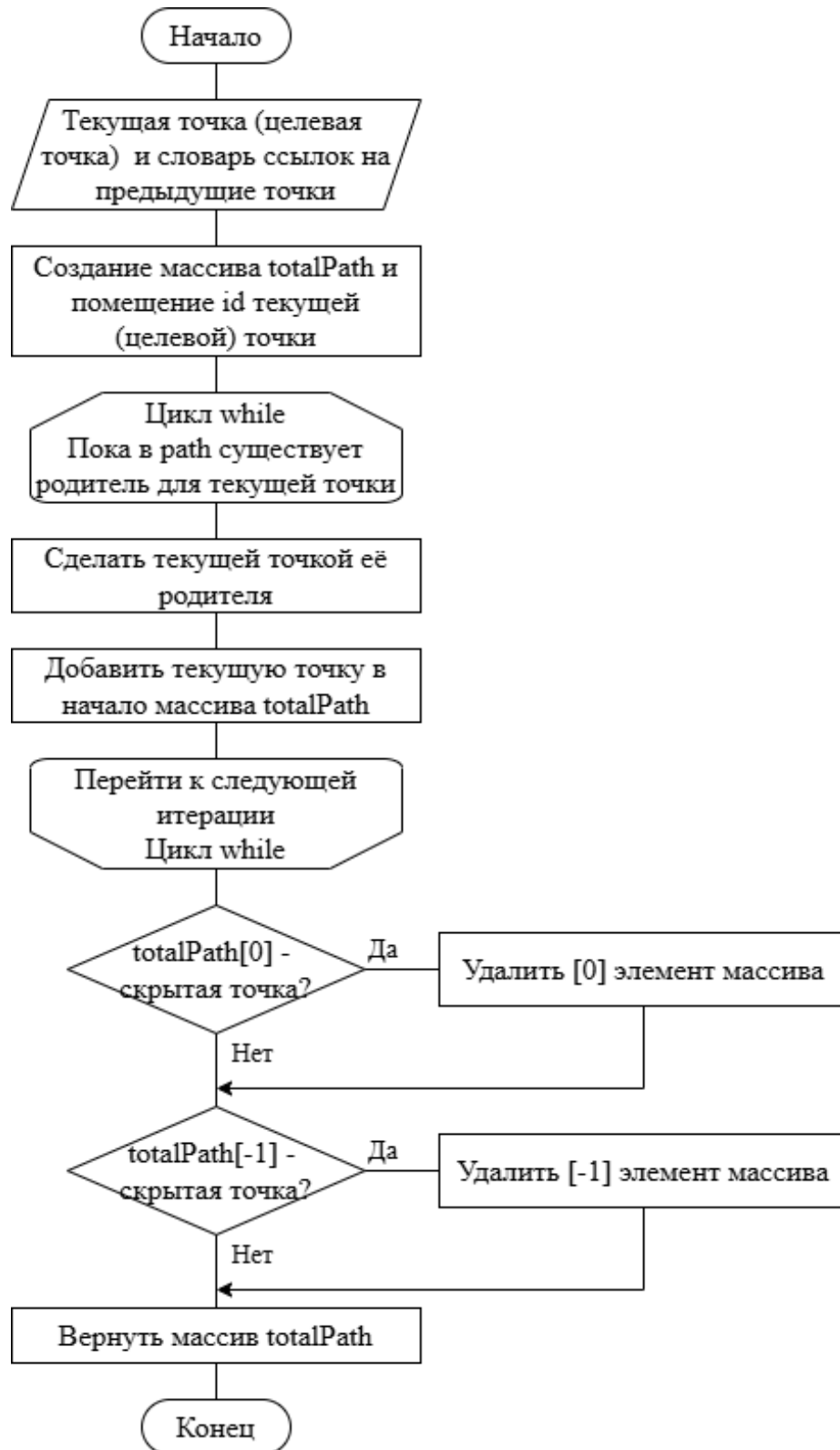


Рисунок 3.3.4 – Схема алгоритма восстановления маршрута

Алгоритм восстановления маршрута начинает работу с того, что в массив итогового пути помещается идентификатор целевой точки. Затем начинает отрабатывать цикл «while».

В теле цикла текущая точка заменяется на своего родителя, а идентификатор новой текущей точки добавляется в начало массива итогового пути. Когда для текущей точки в словаре «path» не найдётся родителя, тогда цикл завершает свою работу. После этого выполняется две проверки на то, является ли первый и/или последний элемент массива скрытой точкой, если это так, то такая точка будет удалена.

После вышеперечисленных действий массив итогового пути возвращается как результат.

Последняя важная функция, которую необходимо рассмотреть – это функция поиска помещения.

Схема алгоритма представлена на рисунке 3.3.5. Он достаточно линейен и не содержит множества условий.

Функция поиска помещения вызывается в нескольких случаях – если было совершено двойное нажатие по двери, если пользователь успешно нашёл дверь с помощью функции поиска помещения, если начальная и конечная точки при построении маршрута совпадают.

Во всех этих случаях происходит обновление поля ввода, сохранение идентификатора двери и отображение панели со всем известной информации об искомом помещении.

Кроме того, если камера не находится на этаже нужной двери, то к нему осуществляется переход.

После этого активируется кнопка нужного этажа, останавливается предыдущая анимация пульсации и подсветка и устанавливается цвет свечения материала объекта для найденной двери, затем запускается анимация пульсации. Если флаг перехода к двери имеет значение «true», то осуществляется перенос камеры.

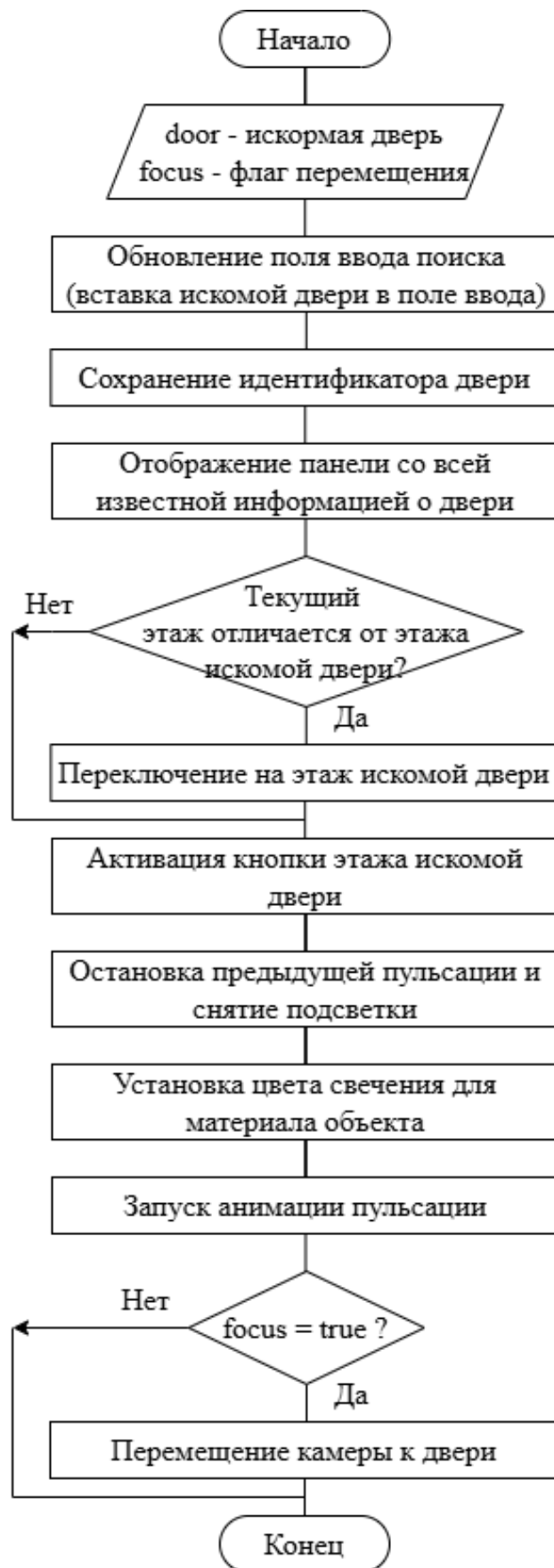


Рисунок 3.3.5 – Схема алгоритма поиска помещения

3.5 Проектирование интерфейсов

Сервис навигации будет иметь две страницы: выбор корпуса и его отображение.

Макет страницы выбора корпуса представлен на рисунке 3.4.1.



Рисунок 3.4.1 – Макет страницы выбора корпуса

В верхней части страницы, предназначенной для выбора корпуса будет располагаться название онлайн-сервиса, а также кнопки переключения между русским и английским языками. Ниже будет располагаться список возможных корпусов.

После выбора корпуса, пользователь будет перемещён на страницу отображения корпуса.

Макет страницы представлен в двух вариантах: для мобильных телефонов и для ПК на рисунках 3.4.2 и 3.4.3 соответственно.

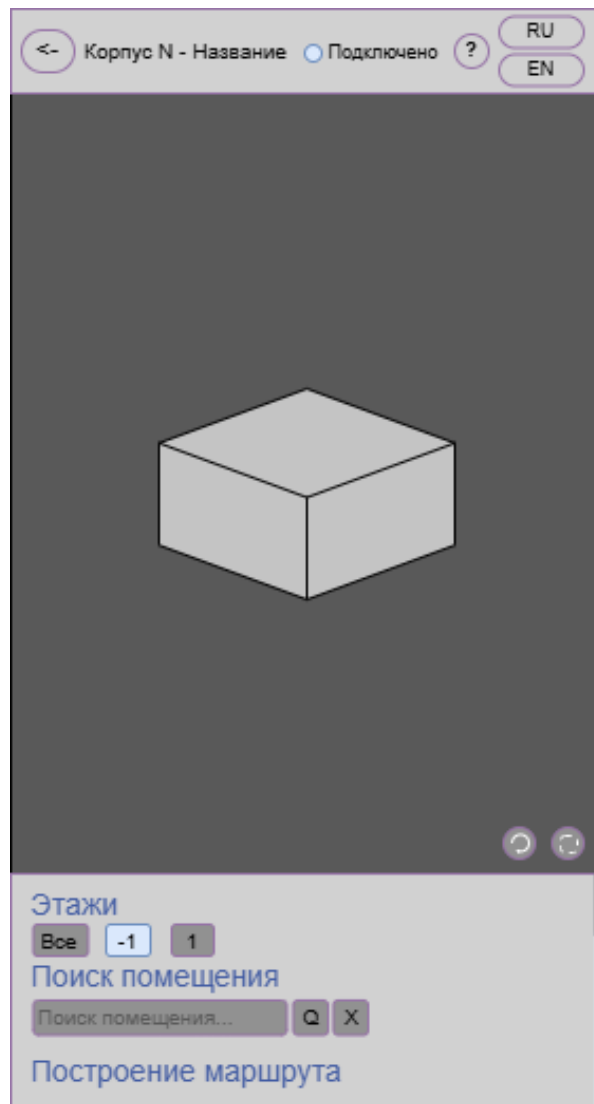


Рисунок 3.4.2 – Макет страницы отображения корпуса для мобильных телефонов

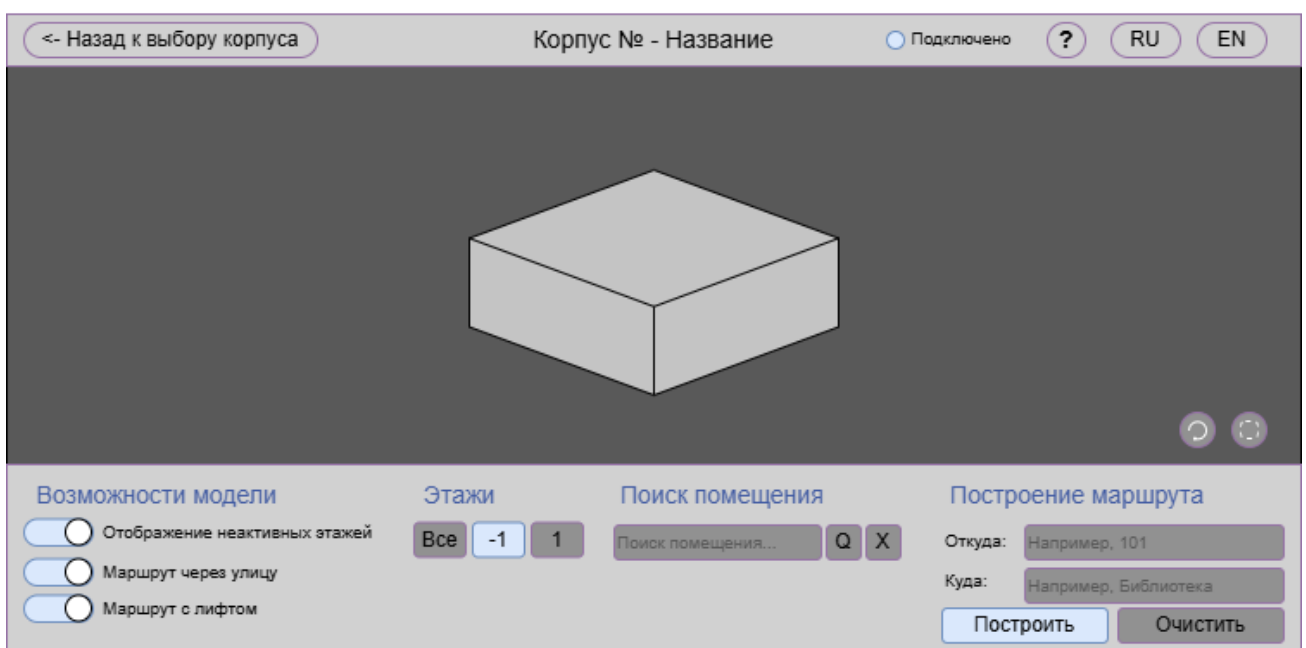


Рисунок 3.4.3 – Макет страницы отображения корпуса для ПК

В верхней панели будет находиться информация (номер и название) выбранного корпуса, а также статус подключения, кнопка возврата к выбору корпуса, кнопка отображение подсказок-инструкций и кнопки смены языка интерфейса.

В окне просмотра будет отображаться сам корпус, а также кнопка сброса вида, перемещающая камеру отображения на изначальную позицию, и кнопка открытия полноэкранного режима.

В нижней панели будут представлены переключатели, помогающие включить или отключить такие возможности как отображение неактивных этажей, прокладывание маршрута через улицу и через лифт. Также здесь находятся кнопки переключения этажей. Кроме того, в этой панели находится поле ввода для поиска помещения, кнопки для поиска и очистки поля, а также поля для построения маршрута, кнопки построения маршрута и очистки полей.

Такой интерфейс позволяет пользователю эффективно взаимодействовать с системой, сводя к минимуму время, необходимое на поиск нужных функций. Кроме того, интерфейс обеспечивает комфортную работу как на мобильных телефонах, так и на персональных компьютерах.

4 РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ОТЛАДКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1 Реализация компонентов программного обеспечения

После завершения этапа проектирования, на котором были определены архитектура системы, спроектированы компоненты и структуры данных, алгоритмы и продуман интерфейс, можно приступать к реализации, тестированию и отладки разрабатываемого онлайн-сервиса.

Клиентская часть онлайн-сервиса для навигации по корпусам ОГУ реализована на языке JavaScript с применением библиотеки Three.js для 3D-визуализации. Библиотека предоставляет высокоуровневый абстрактный интерфейс над низкоуровневым API WebGL, упрощая разработку трёхмерных приложений.

При разработке онлайн-сервиса были использованы различные компоненты библиотеки Three.js, описанные ниже [12].

Сцена (Scene) создаётся строкой `«this.scene = new THREE.Scene()»` и является контейнером, который содержит все 3D-объекты, такие как модель корпуса, камера и источники света. В проекте сцена служит главным элементом для всей 3D-графики.

Перспективная камера создаётся строкой `«new THREE.PerspectiveCamera(fov, aspect, near, far)»` с параметрами угла обзора (fov), соотношения сторон (aspect), ближайшей плоскости (near) и дальней плоскости (far). Камера имитирует зрение человека, благодаря чему объекты, расположенные дальше, выглядят меньше.

Рендер в проекте создаётся строкой `«new THREE.WebGLRenderer({ canvas })»` и отвечает за отображение сцены на HTML-элементе `<canvas>`.

С помощью дополнительного модуля OrbitControls обеспечивается интерактивное управление камерой. Для него настроены различные параметры, такие как вращение сцены с помощью левой кнопкой мыши, масштабирование с помощью колеса мыши, перемещение при помощи средней кнопки мыши и так

далее. Кроме того, для удобства пользователя, настроен параметр `maxPolarAngle`, который запрещает уходить камере под землю.

Освещение в проекте задаётся тремя типами: окружающий свет, который обеспечивает базовое равномерное освещение всех объектов и предотвращает полную темноту и направленный свет, который создаёт объём.

Загрузчик моделей использует `GLTFLoader`, обеспечивая загрузку 3D-моделей в формате `glb`.

Луч необходим для определения того, на какой именно объект наведён курсор мыши. Это необходимо для изменения вида курсора при наведении на дверь, а также при обработке клика по двери для отображения информации о помещении.

Геометрические примитивы используются для создания маркеров начальной и конечной точки маршрута, а также для построения ломаной линии маршрута.

Реализация клиентской части онлайн-сервиса построена на модульном принципе, где каждый компонент отвечает за свой функционал. Модуль `main` обеспечивает работу главной страницы, загружая список корпусов, отображая их карточки. Модуль `viewer` отвечает за просмотр 3D-модели и обрабатывает пользовательские действия. Модель `SceneManager` настраивает камеру и освещение, визуализирует маршруты. `ModelLoader` реализует загрузку и кэширование 3D-моделей в формате `GLB`, исключая повторные сетевые запросы на скачивание корпуса. `ModelGraphBuilder` отвечает за построение графа путевых точек, он загружает файлы `Floor_areas.txt`, `AreaN.txt`, `Doors_metadata.json`, `Points_tags.txt` и формирует связи между точками. `NavigationGraph` хранит граф связей и реализует A^* алгоритм поиска кратчайшего маршрута между двумя помещениями. Модуль `translations` обеспечивает локализацию онлайн-сервиса, предоставляя возможность выбора русского или английского языка.

За реализацию серверной части отвечает файл `server.js`, который обеспечивает раздачу статических файлов (включая 3D-модели, вспомогательные текстовые файлы и JSON) и предоставляет `api` для получения списка корпусов.

4.2 Реализация интерфейса пользователя

Интерфейс пользователя разработан в соответствии с макетами, созданными на этапе проектирования.

Главная страница представляет собой сетку, состоящую из карточек корпусов. Каждая карточка содержит изображение, номер корпуса, его название и адрес. При нажатии на карточку осуществляется переход к просмотру 3D-модели. Пример интерфейса на мобильных телефонах представлен на рисунке 4.2.1.

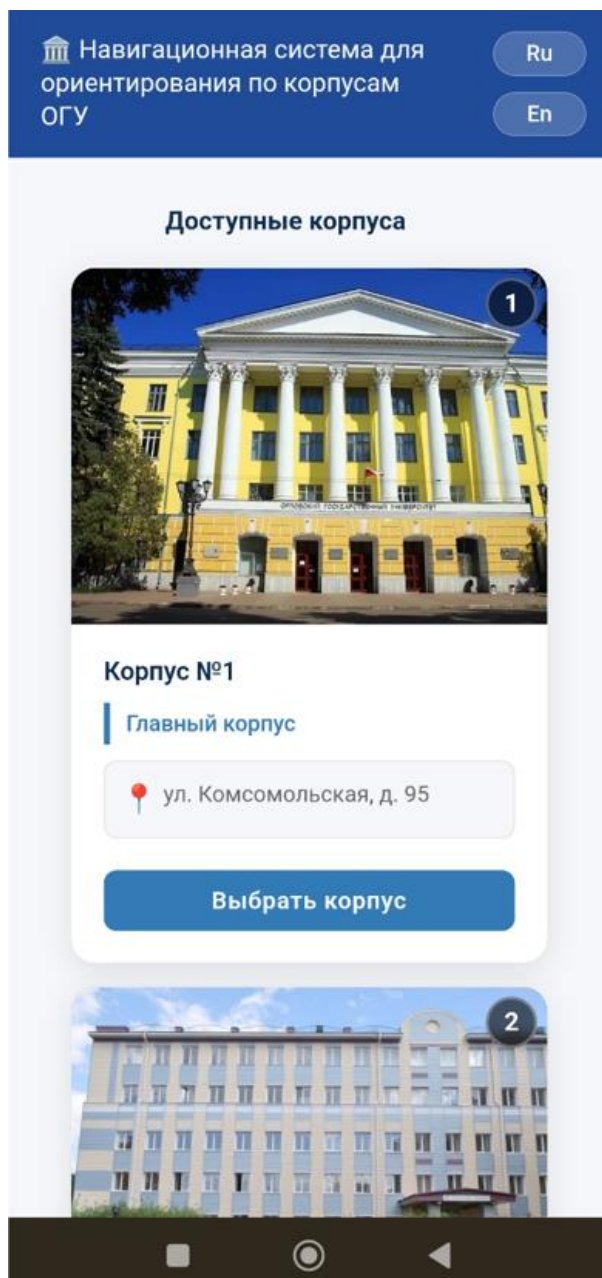


Рисунок 4.2.1 – Страница выбора корпуса для мобильных телефонов

Интерфейс для ПК представлен на рисунке 4.2.2. Его отличие от мобильного заключается в том, что при наведении курсора на кнопки и карточки, происходит

анимация приподнимания этих объектов, что является плюсом для визуального восприятия.

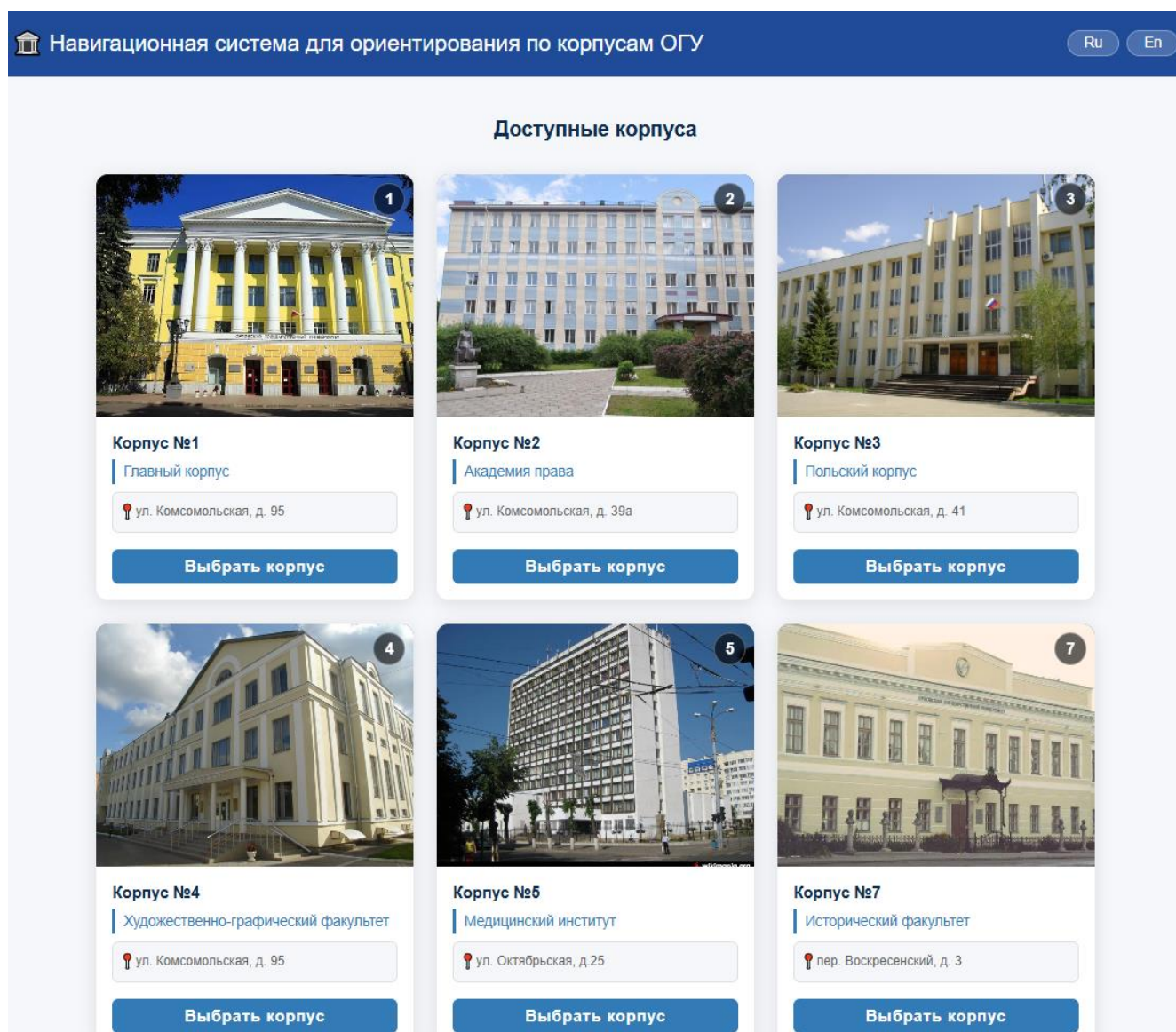


Рисунок 4.2.2 – Страница выбора корпуса для ПК

Интерфейс страницы просмотра модели разделён на 3 функциональные зоны, как представлено на рисунке 4.2.3.

Верхняя панель содержит кнопку возврата к списку корпусов, название выбранного корпуса, индикатор подключения, кнопку, выводящую подсказку-инструкцию, а также кнопки переключения языка (Ru и En).

Центральная область представляет собой окно 3D-визуализации с элементами управления камерой, а также кнопками сброса вида и открытия полноэкранного режима.

Нижняя панель предназначена для переключения этажей, управления возможностями модели (переключение отображения невыбранных этажей), а также же для поиска помещения и построения маршрута.



Рисунок 4.2.3 – Страница просмотра модели корпуса для мобильных телефонов

Интерфейс страницы просмотра 3D модели для ПК представлен на рисунке 4.2.4. Он слабо отличается от интерфейса для мобильных телефонов. Разница состоит в том, что нижняя панель не имеет вертикальной прокрутки, так как полностью помещается в экран.

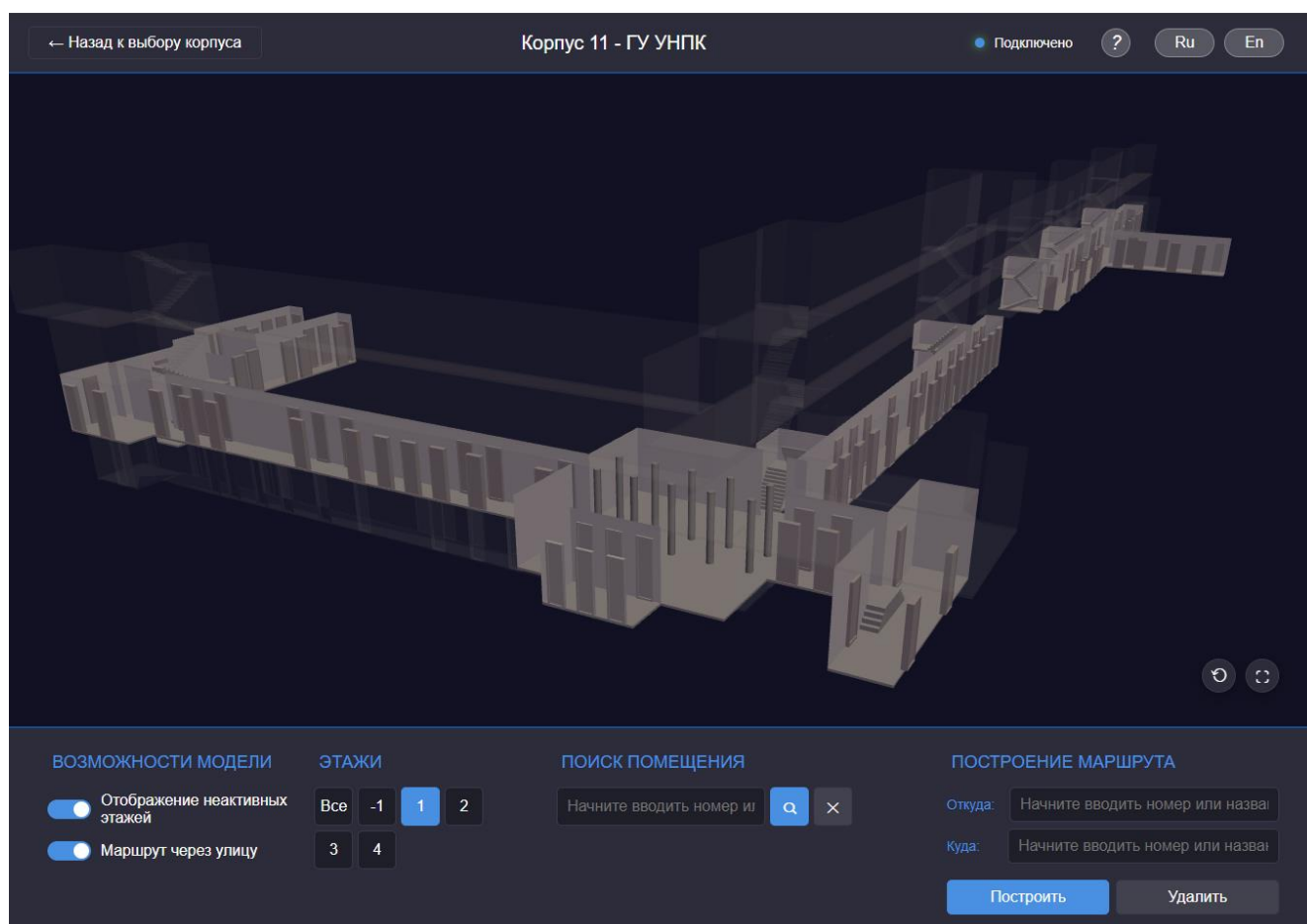


Рисунок 4.2.3 – Страница просмотра модели корпуса для ПК

4.3 Тестирование и отладка программного обеспечения

Для того, чтобы проверить корректность работы онлайн-сервиса, было принято решение провести функциональное тестирование [13]. Оно необходимо для проверки того, как работает программное обеспечение в соответствии с заданными требованиями. Главная цель функционального тестирования – убедиться, что все функции программы выполняются так, как ожидалось.

Было проведено тестирование следующих модулей:

- главная страница;
- загрузка 3D-модели;
- переключение языка;
- управление камерой;
- управление этажами;
- поиск помещения;

- построение маршрута;
- маршрутные опции.

Таблица 4.3.1 – Результаты функционального тестирования главной страницы

№	Действие	Ожидаемый результат	Правильность работы
1	Загрузка страницы	Отображается список корпусов	Полная
2	Клик по карточке корпуса или кнопке Выбор корпуса"	Переход на страницу просмотра	Полная

Таблица 4.3.2 – Результаты функционального тестирования загрузки 3D-модели

№	Действие	Ожидаемый результат	Правильность работы
1	Переход на viewer.html?id=корректный_номер	Отображается индикатор загрузки; модель успешно загружается и отображается; индикатор исчезает	Полная
2	Переход на viewer.html?id=некорректный_номер	Выполняется пересылка на главную страницу	Полная
3	Повторный просмотр того же корпуса	Модель загружается из кэша (ModelLoader.cache), повторного запроса к серверу нет	Полная
4	Отсутствие поддержки WebGL	Выводится сообщение об ошибке и выполняется пересылка на главную страницу	Полная

Таблица 4.3.3 – Результаты функционального тестирования переключения языка

№	Действие	Ожидаемый результат	Правильность работы
1	Нажата кнопка "Ru"	Все тексты интерфейса переключаются на русский	Полная

Продолжение таблицы 4.3.3

№	Действие	Ожидаемый результат	Правильность работы
2	Нажата кнопка "En"	Все тексты интерфейса переключаются на английский	Полная
3	Переход на другую страницу или обновление текущей	Сохранение того языка, который был выбран	Полная
4	Смена языка при активном маршруте или найденном помещении	Текст кнопок и подсказок обновляется, маршрут и описания не сбрасываются	Полная

На рисунках 4.3.1 и 4.3.2 продемонстрирован интерфейс на английском языке для мобильных телефонов и ПК.

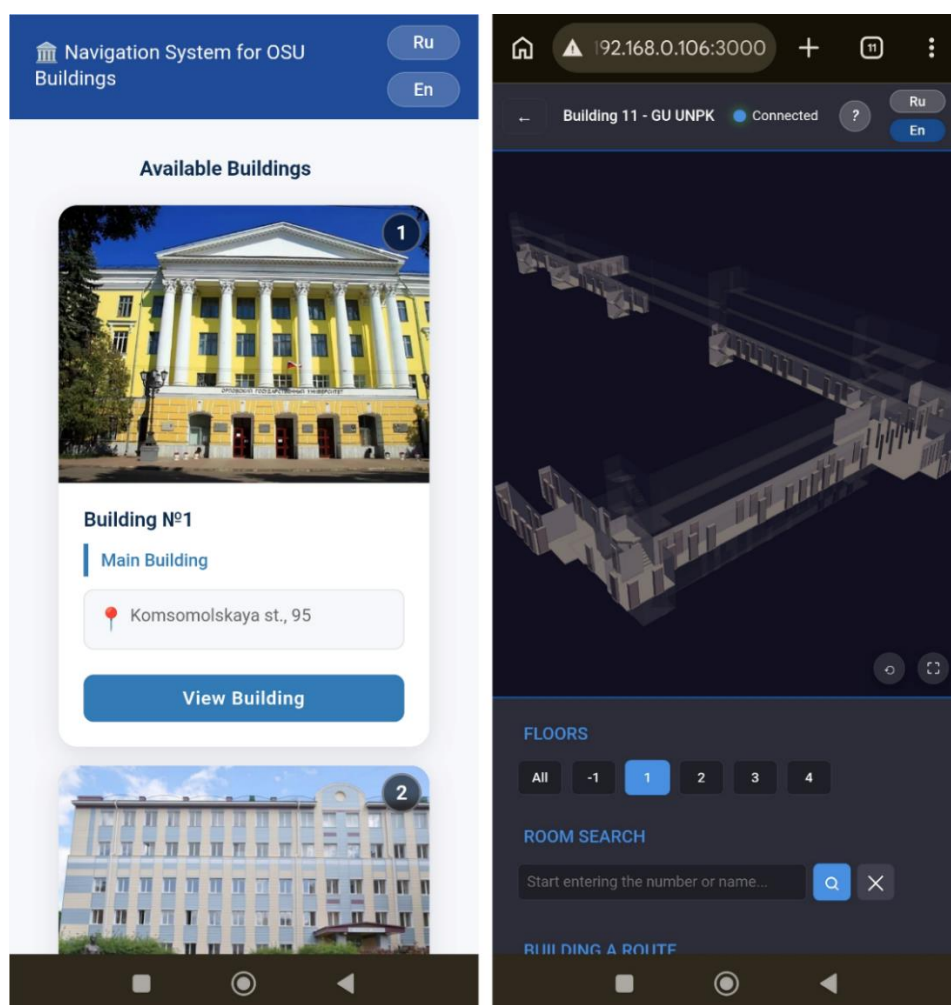


Рисунок 4.3.1 – Интерфейс на английском языке для мобильного телефона

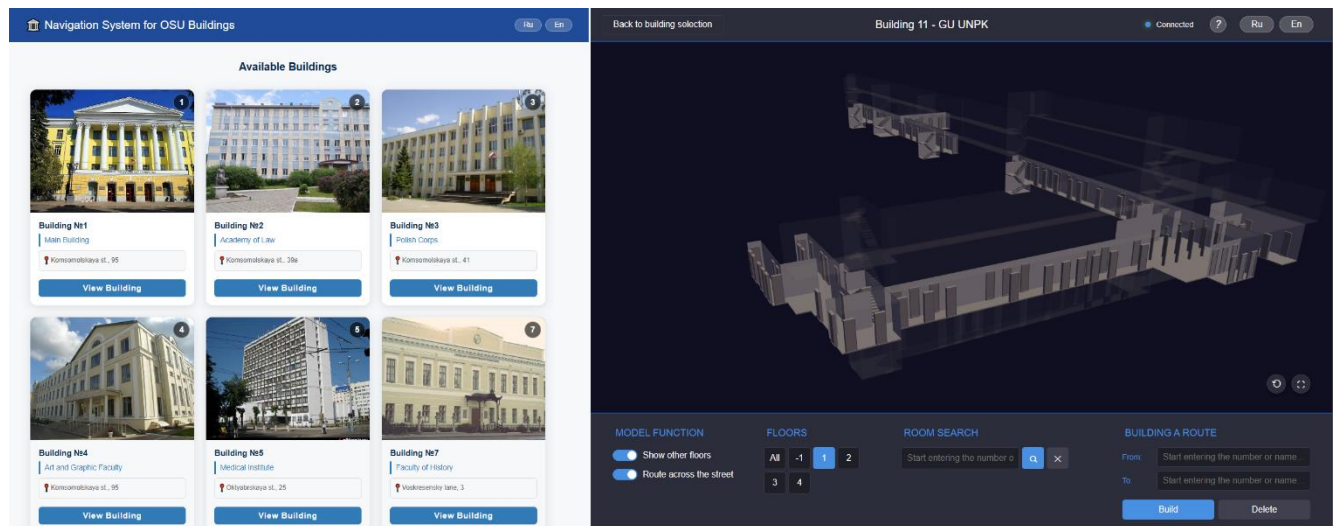


Рисунок 4.3.2 – Интерфейс на английском языке для ПК

Таблица 4.3.4 – Результаты функционального тестирования управления камерой

№	Действие	Ожидаемый результат	Правильность работы
1	Вращение левой кнопкой мыши (ЛКМ) на ПК	Модель вращается вокруг центра сцены	Полная
2	Перемещение средней кнопкой мыши (СКМ) на ПК	Модель перемещается по экрану	Полная
3	Масштабирование колесом мыши на ПК	Модель приближается и отдаляется	Полная
4	Один нажатый палец перемещается на сенсорном экране	Модель вращается	Полная
5	Два нажатых пальца перемещаются по сенсорному экрану	Модель перемещается	Полная
6	Два нажатых пальца соединяются/разъединяются на сенсорном экране	Модель масштабируется	Полная

Продолжение таблицы 4.3.4

№	Действие	Ожидаемый результат	Правильность работы
7	Нажатие кнопки сброса вида (☺)	Камера возвращается в исходное положение	Полная
8	Нажатие кнопки полноэкранного режима (⌘)	Переключение в полноэкранный режим и обратно	Полная
9	Попытка наклонить камеру ниже уровня земли	Камера не уходит под землю (ограничение $\max \text{PolarAngle} = \pi/2$)	Полная

Таблица 4.3.5 – Результаты функционального тестирования управления этажами

№	Действие	Ожидаемый результат	Правильность работы
1	Нажатие кнопки "Все"	Показываются все этажи, все объекты видимы	Полная
2	Нажатие кнопки этажа (например, "1")	Показывается только этаж 1, остальные полупрозрачны (если включён режим прозрачности) или полностью скрыты (если выключен)	Полная
3	Переключение между этажами	Активная кнопка подсвечивается, этаж меняется согласно нажатой кнопке	Полная
4	Включение тумблера "Показывать другие этажи"	Неактивные этажи становятся полупрозрачными	Полная
5	Выключение тумблера "Показывать другие этажи"	Неактивные этажи полностью скрываются	Полная

На рисунке 4.3.3 представлены различные комбинации вариантов отображения этажей: выбран только один этаж, переключатель «Показывать другие этажи» включён; выбран только один этаж, переключатель «Показывать другие этажи» выключен, выбраны все этажи. Пример представлен для мобильных телефонов.

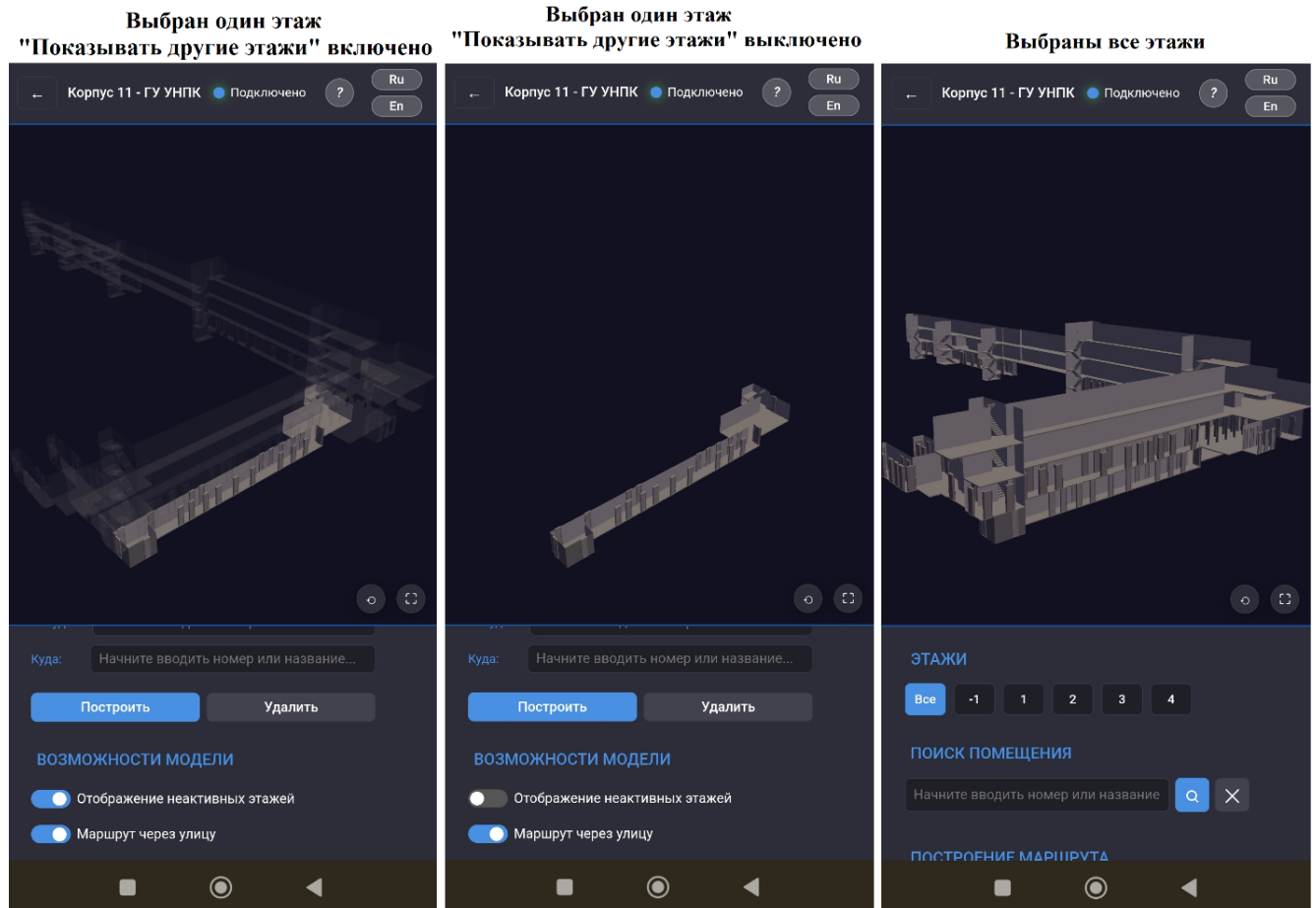


Рисунок 4.3.3 – Комбинации вариантов отображения этажей для мобильных телефонов

Таблица 4.3.6 – Результаты функционального тестирования поиска помещения

№	Действие	Ожидаемый результат	Правильность работы
1	Ввод части номера или описания (например, «31»)	Появляется выпадающий список подходящих дверей с номерами, описаниями и этажами	Полная
2	Выбор варианта из выпадающего списка	Поле заполняется выбранным значением	Полная
3	После выбора варианта из выпадающего списка нажата кнопка "Найти"	Камера перемещается на искомую дверь, этаж переключается на этаж двери. Дверь подсвечивается и начинает пульсировать. В панели информации отображается этаж, номер и описание двери.	Полная

Продолжение таблицы 4.3.6

№	Действие	Ожидаемый результат	Правильность работы
4	Нажатие кнопки "Очистить"	Поле очищается, подсветка и пульсация снимаются, панель информации скрывается, фильтр выпадающего списка сбрасывается	Полная
5	Двойной клик (тап) по двери в 3D-сцене	Поле поиска заполняется. Дверь подсвечивается и начинает пульсировать. В панели информации отображается этаж, номер и описание двери.	Полная
6	Поиск по описанию (например, "гардероб" или номеру (например, "183"))	Дверь с соответствующим номером или описанием находится и отображается аналогично поиску через выбор из выпадающего списка	Полная
7	Поиск несуществующей аудитории	Выпадающий список не показывает вариантов	Полная
8	Нажатие кнопки "Найти" при поиске несуществующей аудитории	Ничего не найдёт, отобразится сообщение о том, что помещение не найдено	Полная
9	Повторный поиск помещения	Старая подсветка снимается, пульсация останавливается, начинаются действия, которые происходят при поиске помещения	Полная

На рисунках 4.3.4 и 4.3.5 изображён процесс поиска помещения для мобильных телефонов и ПК, показан выпадающий список и визуальные эффекты при найденном помещении.

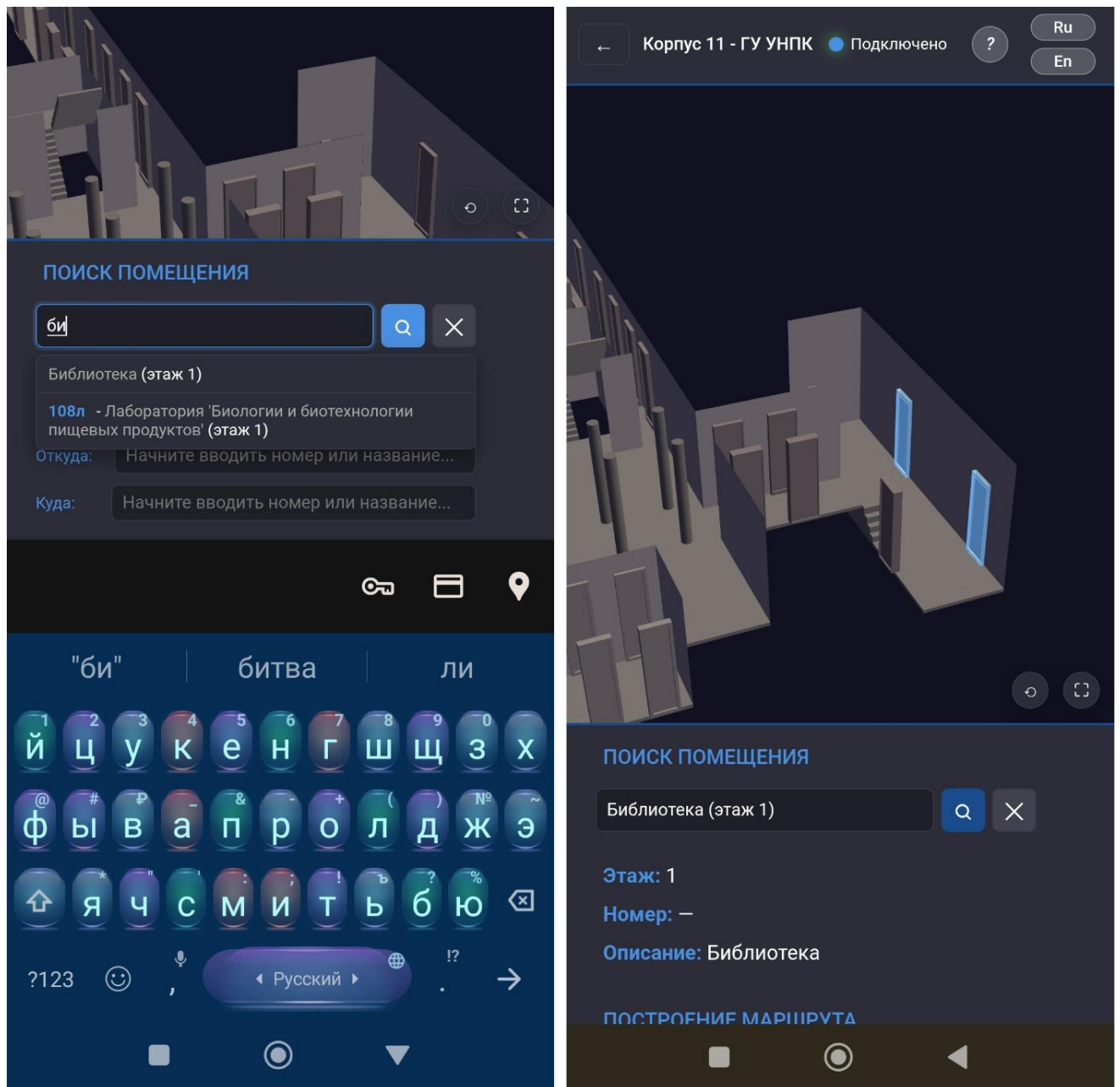


Рисунок 4.3.4 – Поиск помещения для мобильных телефонов

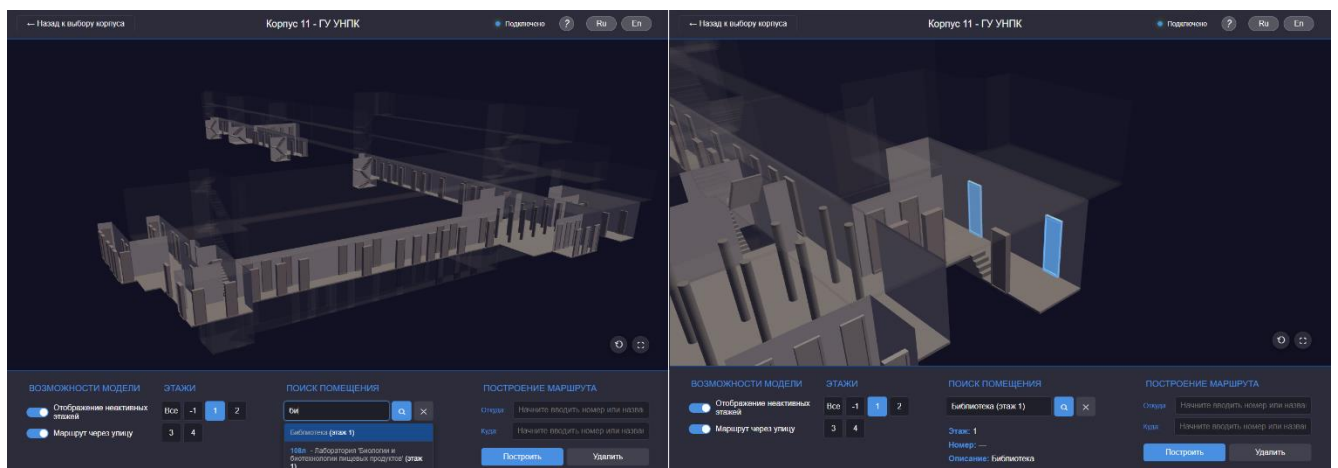


Рисунок 4.3.5 – Поиск помещения для ПК

Как видно из рисунков, при найденном помещении происходит перемещение камеры к искомой двери и подсветка (анимация пульсации не заметна на скриншотах экрана).

Таблица 4.3.7 – Результаты функционального тестирования построения маршрута

№	Действие	Ожидаемый результат	Правильность работы
1	Выбор начальной точки через выпадающий список	Поле заполняется, ID двери сохраняется	Полная
2	Выбор конечной точки через выпадающий список	Поле заполняется, ID двери сохраняется	Полная
3	Нажатие кнопки "Построить маршрут" при правильно заполненных точках, и существующем маршруте	На сцене отображается линия пути; начальная и конечная точки отмечены сферами; камера фокусируется на начальной двери; этаж начальной точки автоматически выбирается	Полная
4	Нажатие кнопки "Построить маршрут" при правильно заполненных точках, но несуществующем маршруте	Стирается старый маршрут, если есть; пользователь получает сообщение о том, что маршрут не найден	Полная
5	Попытка построить маршрут без выбора обеих точек	Появление всплывающего сообщения с просьбой выбрать точки	Полная
6	Повторное построение маршрута	Старый маршрут удаляется, начинаются действия, происходящие при построении маршрута	Полная
7	Нажатие кнопки "Очистить"	Поля «Откуда» и «Куда» очищаются; маршрут и маркеры удаляются; информация о маршруте скрывается	Полная

На рисунках 4.3.6 и 4.3.7 для мобильных телефонов и ПК изображён найденный между помещениями.



Рисунок 4.3.6 – Поиск маршрута для мобильных телефонов

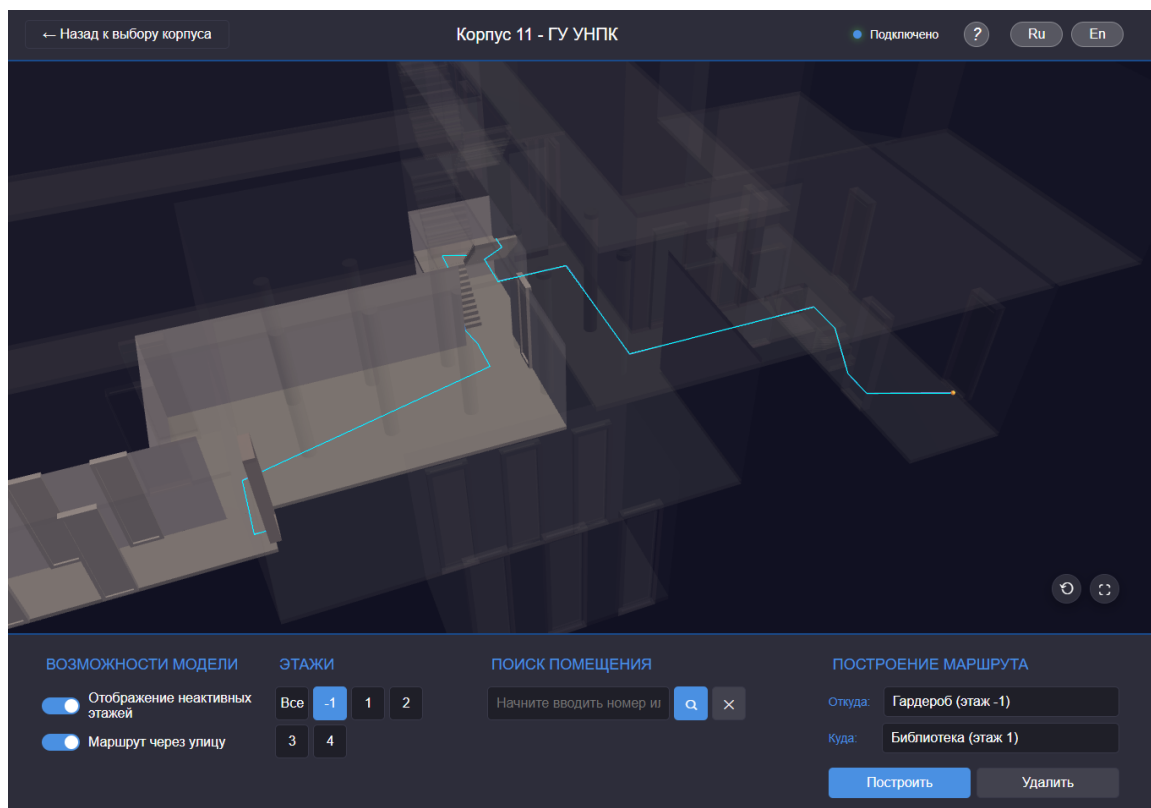


Рисунок 4.3.7 – Поиск маршрута для ПК

Как видно из рисунка, линия маршрута видна на всех этажах вне зависимости от того, какой из них выбран активным.

Таблица 4.3.8 – Результаты функционального тестирования маршрутных опций

№	Действие	Ожидаемый результат	Правильность работы
1	Отсутствие уличных точек в корпусе	Переключатель "Маршрут через улицу" не отображается в панели	Полная
2	Отсутствие точек лифта в корпусе	Переключатель "Маршрут с лифтом" не отображается в панели	Полная
3	Выключение тумблера «Маршрут через улицу» (при наличии уличных точек в корпусе)	Построенный маршрут не будет использовать точки с флагом isStreet	Полная
4	Выключение тумблера «Маршрут с лифтом» (при наличии точек лифта)	Построенный маршрут не будет использовать точки с флагом isElevator	Полная
5	Изменение состояния любого переключателя после построения маршрута	Маршрут перестраивается с учётом новых опций	Полная

На рисунках 4.3.8, 4.3.9 и 4.3.10 для мобильных телефонов и ПК изображён процесс поиска маршрута между помещениями, для которых кратчайшим путём будет путь через улицу. На изображениях представлено 2 варианта поиска: при включённом и при выключенном переключателе «Маршрут через улицу».

При изменении состояния переключателя происходит перестроение маршрута с учётом новых условий.

Кроме того, на рисунках отсутствует переключатель «Маршрут с лифтом», поскольку данный корпус не содержит точек, проходящих через лифт (нет лифта).

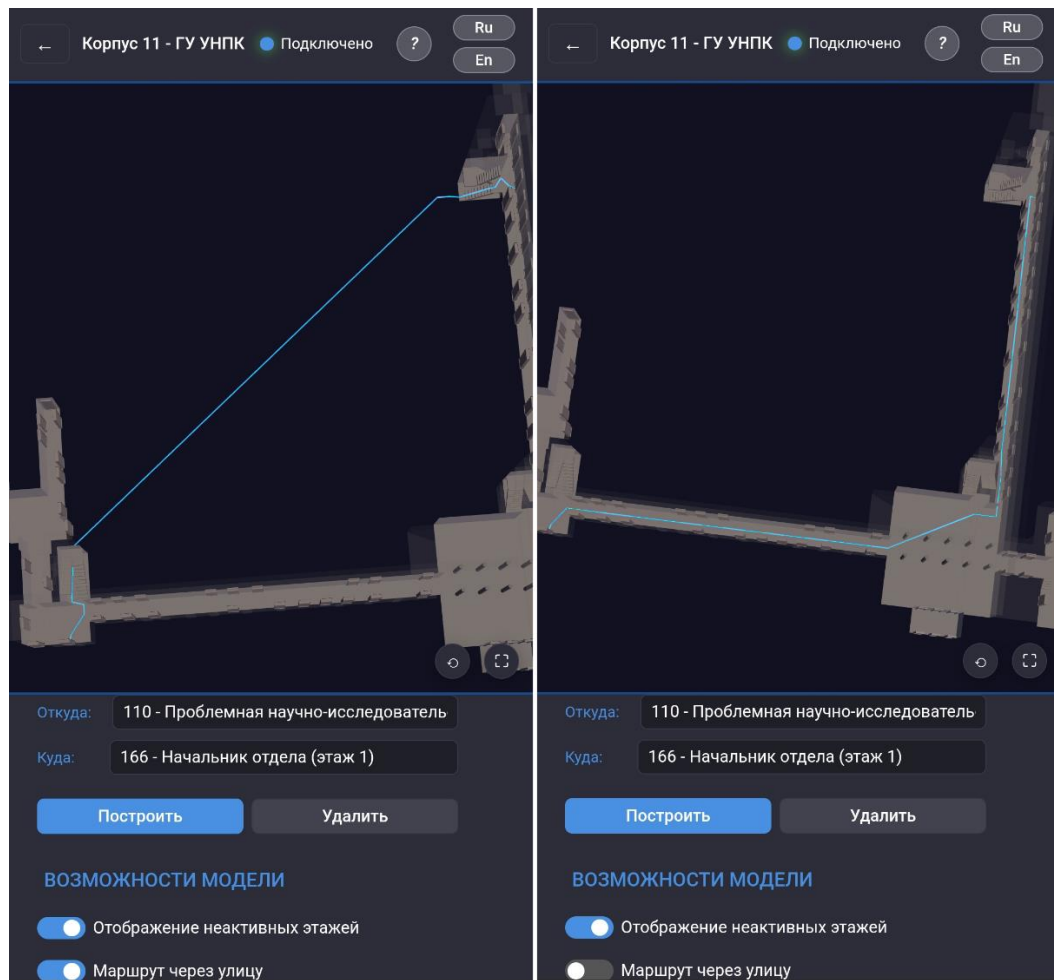


Рисунок 4.3.8 – Поиск маршрута с разными состояниями переключателя для мобильных телефонов

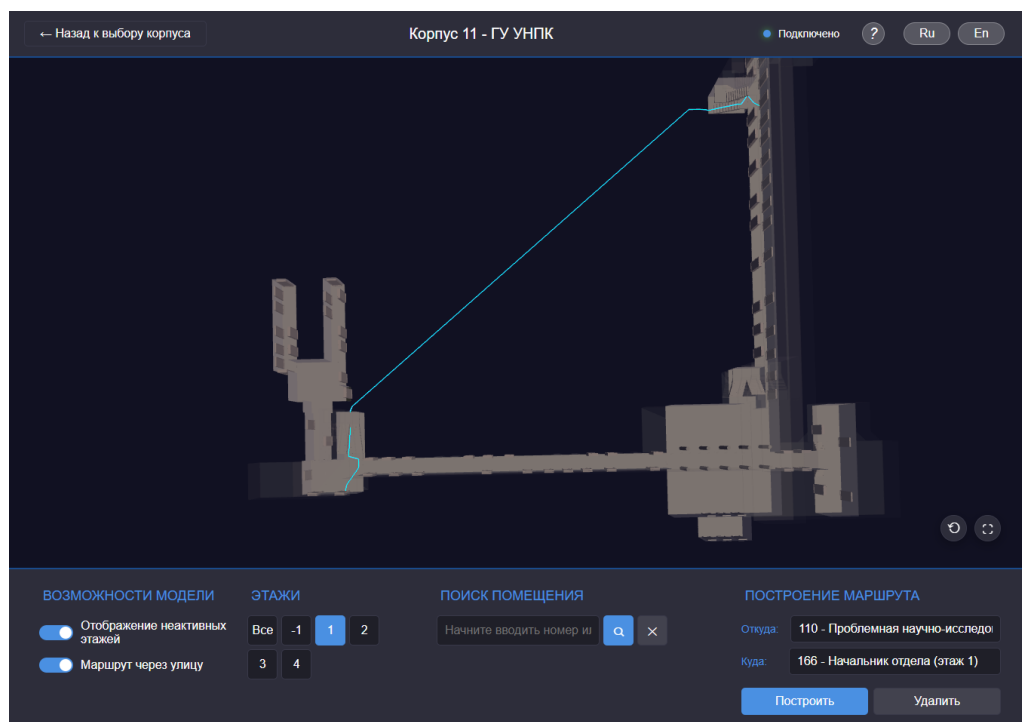


Рисунок 4.3.9 – Поиск маршрута с возможностью проходить через улицу для ПК

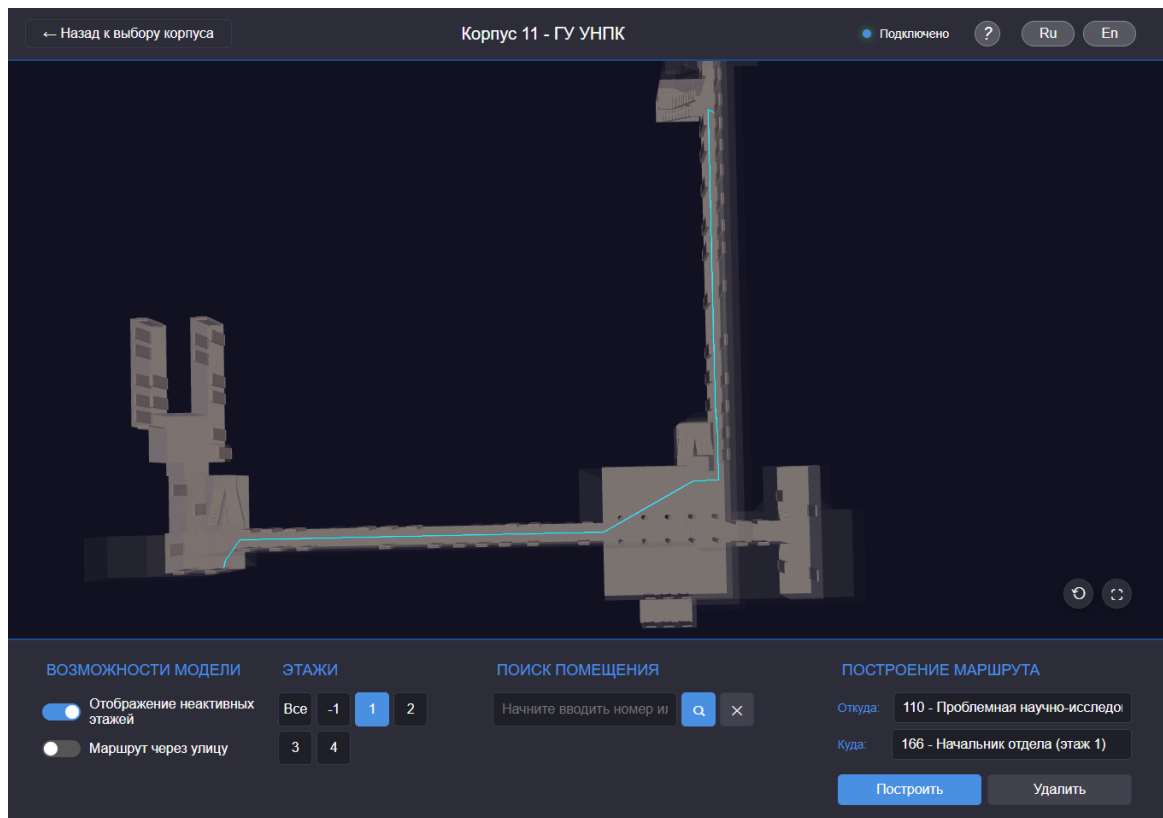


Рисунок 4.3.10 – Поиск маршрута без возможности проходить через улицу для ПК

В ходе тестирования были проверены как основные сценарии использования, направленные на проверку функционирования онлайн-сервиса при корректных входных данных, так и альтернативные, проверяющие поведение системы в случае некорректного ввода данных, отсутствия необходимых файлов, отсутствия поддержки WebGL, поиск несуществующей аудитории, попытка построить маршрут без выбора точек и так далее.

В заключение функционального тестирования можно сказать, что разработанный онлайн-сервис для ориентирования по корпусам университета полностью соответствует функциональным требованиям. Все ключевые возможности работают корректно и без ошибок.

ЗАКЛЮЧЕНИЕ

Система навигации внутри зданий университета обладает большим потенциалом для дальнейшего развития. Переход к 3D-модели помог создать более наглядный, удобный и запоминающийся инструмент для ориентирования. Однако есть и другие способы улучшения системы: интеграция с другими информационными системами университета, такими как расписание занятий или мероприятий, что помогло бы быстрее искать свободные помещения или видеть, когда освободится какая-то конкретная аудитория; персонализировать интерфейс, который позволит учитывать особенности человека и предоставлять маршруты в соответствии с его особенностями, например, маршруты для маломобильных граждан или служебные коридоры; собирать и анализировать обратную связь, улучшая систему для большего удобства пользователей.

В ходе выполнения выпускной квалификационной работы был разработан онлайн-сервис для ориентирования по корпусам университета на базе WebGL.

В результате анализа были сформулированы требования, определяющие перечень основных функций системы. Так же разработаны спецификации, которые послужили основой для проектирования классов, модулей и интерфейсов взаимодействия между клиентом и сервером.

На этапе проектирования онлайн-сервиса были определены правила построения графа навигации, установлены форматы входных файлов и выходных данных, продумана система переходов между состояниями, которая обеспечивает свободную навигацию пользователя между всеми функциональными блоками интерфейса.

В ходе реализации был создан онлайн-сервис навигации, полностью соответствующий всем требованиям. С помощью функционального тестирования была проверена корректность работы системы в различных сценариях использования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. 3D экскурсии – Текст. Изображение : электронные // ФГБОУ ВО «ОГУ имени И.С. Тургенева» : [сайт]. – URL: https://oreluniver.ru/virt_towr (дата обращения 27.11.2025).
2. Интерактивная карта кампусов РТУ МИРЭА – Текст : электронный // МИРЭА – Российский технологический университет : [сайт]. – URL: <https://map.kkmbrr.ru/> (дата обращения 03.11.2025).
3. Интерактивная карта-навигатор по корпусам ОГУ им. И.С.Тургенева – Текст. Изображение : электронные // ФГБОУ ВО «ОГУ имени И.С. Тургенева» : [сайт]. – URL: <https://map.oreluniver.ru/> (дата обращения 07.11.2025).
4. Indoor Navigation using Augmented Reality / A. Martin [et al.]. – Текст : электронный // ResearchGate : [сайт]. – 2021. – 17 февр. – URL: https://www.researchgate.net/publication/349390904_Indoor_Navigation_using_Augmented_Reality (дата обращения 10.11.2025).
5. Cockburn, A. Revisiting 2D vs 3D Implications on Spatial Memory / A. Cockburn. – Текст : электронный // Canterbury : [сайт]. – URL: <https://www.csse.canterbury.ac.nz/andrew.cockburn/papers/auic-2d3d.pdf> (дата обращения 13.11.2025).
6. Обзор навигационных систем для ориентирования внутри учебных корпусов / М.А. Банных, А.Ю. Ужаринский, С.В. Новиков, А.Е. Мельников // Сборник статей Международной научно-практической конференции / под общей редакцией И. И. Ивановской, Л. А. Кузьмины. – Петрозаводск : МЦНП «НОВАЯ НАУКА», 2026. – С.299–306.
7. Клиент-серверная архитектура. SA для самых маленьких – Текст : электронный // Хабр : [сайт]. – URL: <https://habr.com/ru/articles/836464/> (дата обращения 18.12.2025).
8. Учебное пособие "Диаграмма развертывания" | Что такое диаграмма развертывания – Текст : электронный // Creately | Visual Collaboration &

Diagramming Platform : [сайт]. – URL: <https://creately.com/blog/ru/uncategorized-ru/учебное-пособие-по-диаграмме-развёрт/> (дата обращения 24.12.2025).

9. Четыре Типа Отношений В Диаграмме Вариантов Использования – Текст : электронный // Visual Paradigm - AI-Powered Visual Modeling Platform : [сайт]. – URL: <https://blog.visual-paradigm.com/ru/the-four-types-of-relationship-in-use-case-diagram/> (дата обращения 13.01.2026).

10. Учебное пособие по диаграмме действий | Как нарисовать диаграмму активности – Текст : электронный // Creately | Visual Collaboration & Diagramming Platform : [сайт]. – URL: <https://creately.com/blog/ru/uncategorized-ru/учебник-по-диаграмме-активности/> (дата обращения 20.01.2026).

11. Коллекции (collections) – Blender LTS Manual – Текст : электронный // Blender Documentation: [сайт]. – URL: https://docs.blender.org/manual/ru/dev/scene_layout/collections/collections.html (дата обращения 10.03.2026).

12. 3D Дизайн на Three.js – Текст. Изображение : электронные // itProger : [сайт]. – URL: <https://itproger.com/ua/course/three-js> (дата обращения 24.03.2026).

13. Что такое функциональное тестирование? – Текст : электронный // Центр тестирования и развития ПО компании IBS: [сайт]. – URL: <https://ibs-qa.ru/media/chto-takoe-funktsionalnoe-testirovanie/> (дата обращения 12.05.2026).

Фрагменты исходного кода**NavigationGraph.js** (Класс Point и его наследники)

```
class Point {
  constructor(id, modelObject, parentArea) {
    this.id = id;
    this.modelObject = modelObject;
    this.parentArea = parentArea;
    this.connections = [];
    this.position = modelObject ? modelObject.position.clone() : null;
    this.isStreet = false;
    this.isElevator = false;
    this.isHidden = false;
  }

  addConnection(targetPoint) {
    if (!this.connections.includes(targetPoint))
      this.connections.push(targetPoint);
  }

  getAllConnections() {
    return this.connections;
  }

  getEuclideanDistance(targetPoint) {
    const dx = this.position.x - targetPoint.position.x;
    const dy = this.position.y - targetPoint.position.y;
    const dz = this.position.z - targetPoint.position.z;

    return Math.sqrt(dx*dx + dy*dy + dz*dz);
  }
}

class PrimaryPoint extends Point {
  constructor(id, modelObject, parentArea) {
    super(id, modelObject, parentArea);
    this.type = 'primary';
  }
}

class SecondaryPoint extends Point {
  constructor(id, modelObject, parentArea) {
    super(id, modelObject, parentArea);
    this.type = 'secondary';
  }
}
```

```

class DoorPoint extends Point {
  constructor(id, modelObject, parentArea) {
    super(id, modelObject, parentArea);
    this.type = 'door';
    this.number = "";
    this.descriptionRu = "";
    this.descriptionEn = "";
    this.floor = null;
  }

  setInfo(number = "", descriptionRu = "", descriptionEn = "", floorMapping = null) {
    this.number = number;
    this.descriptionRu = descriptionRu;
    this.descriptionEn = descriptionEn;

    // определение этажа из родительской Area
    for (const [floor, areas] of Object.entries(floorMapping)) {
      if (areas.includes(this.parentArea.index)) {
        this.floor = parseInt(floor);
        break;
      }
    }
  }

  getDescription(lang) {
    const description = lang === 'en' ? this.descriptionEn : this.descriptionRu;
    return description;
  }

  getInfo(lang) {
    return {
      id: this.id,
      number: this.number,
      description: this.getDescription(lang),
      floor: this.floor,
      position: this.position,
      area: this.parentArea.index
    };
  }
}

```

NavigationGraph.js (Алгоритм построения маршрута)

```

navigateAStar(startDoorID, endDoorID, options = {}) {
  const { allowStreet = true, allowElevator = true } = options;

  const startDoor = this.getPoint(startDoorID);
  const endDoor = this.getPoint(endDoorID);

  const openSet = new Set();
  const closedSet = new Set();

  const gScore = new Map();

```

```

const fScore = new Map();

const path = new Map();

gScore.set(startDoorID, 0);
fScore.set(startDoorID, startDoor.getEuclideanDistance(endDoor));
openSet.add(startDoor);

while (openSet.size > 0) {
  let current = null;
  let lowestScore = Infinity;

  for (const point of openSet) {
    const fs = fScore.get(point.id);

    if (fs !== undefined && fs < lowestScore) {
      lowestScore = fs;
      current = point;
    }
  }

  if (current.id === endDoorID)
    return this.reconstructPath(path, current);

  openSet.delete(current);
  closedSet.add(current);

  const connections = current.getAllConnections();

  for (const connection of connections) {
    if (!allowStreet && connection.isStreet)
      continue;
    if (!allowElevator && connection.isElevator)
      continue;
    if (closedSet.has(connection))
      continue;

    const distance = current.getEuclideanDistance(connection);
    const estimatedScore = (gScore.get(current.id) !== undefined ? gScore.get(current.id) : 0)
+ distance;

    if (!openSet.has(connection))
      openSet.add(connection);
    else if (gScore.get(connection.id) !== undefined && estimatedScore >=
gScore.get(connection.id))
      continue;

    path.set(connection.id, current);
    gScore.set(connection.id, estimatedScore);
    fScore.set(connection.id, estimatedScore + connection.getEuclideanDistance(endDoor));
  }
}

```

```

    return null;
}
getEuclideanDistance(targetPoint) {
    const dx = this.position.x - targetPoint.position.x;
    const dy = this.position.y - targetPoint.position.y;
    const dz = this.position.z - targetPoint.position.z;

    return Math.sqrt(dx*dx + dy*dy + dz*dz);
}
reconstructPath(path, current) {
    const totalPath = [current.id];

    while (path.has(current.id)) {
        current = path.get(current.id);
        totalPath.unshift(current.id);
    }

    if (this.getPoint(totalPath[0])?.isHidden)
        totalPath.shift();
    if (this.getPoint(totalPath[totalPath.length - 1])?.isHidden)
        totalPath.pop();

    return totalPath;
}

```

DoorSearch.js (Алгоритм поиска пути)

```

handleSearch() {
    this.clearDoorHighlight();
    this.hideInfo();

    const query = this.input?.value.trim();
    if (!query) {
        this.doorInfo.innerHTML = `<div class="door error">${t('selectPointPrompt')}</div>`;
        this.doorInfo.classList.remove('hidden');
        return;
    }

    let door = this.input?.dataset.doorId
        ? window.navGraph?.getPoint(this.input.dataset.doorId)
        : this.findByQuery(query);

    if (door) this.showDoor(door);
    else {
        this.doorInfo.innerHTML = `<div class="door error">${t('doorNotFound')}</div>`;
        this.doorInfo.classList.remove('hidden');
    }
}
findByQuery(query) {
    const searchTerm = query.toLowerCase().trim();

    let exactNumberMatch = null;
    let exactDescriptionMatch = null;

```

```

window.navGraph.getAllDoors().forEach(door => {
  const number = door.number?.toLowerCase() || "";
  const description = door.getDescription(window.currentLanguage)?.toLowerCase() || "";

  if (number === searchTerm) {
    exactNumberMatch = door;
  } else if (description === searchTerm && !exactNumberMatch) {
    exactDescriptionMatch = door;
  }
});

return exactNumberMatch || exactDescriptionMatch || null;
}
showDoor(door, focus = true) {
  this.input.value = this.autocomplete.getDisplayText(door);
  this.input.dataset.doorId = door.id;
  this.showInfo(door);
  this.moveToFloor(door);
  this.highlightDoor(door);

  if (focus) {
    sceneManager.focusOnPoint(door, 20, 5);
  }
}

```

ModelGraphBuilder.js (Загрузка модели и построение графа)

```

async build(model, buildingId) {
  this.graph = new NavigationGraph();
  this.buildingId = buildingId;

  await this.loadFloorMapping();
  this.extractWaypoints(model);
  this.hideNonDoors();
  await this.loadConnections();
  await this.loadDoorInfo();
  await this.loadPointsTags();

  return this.graph;
}

extractWaypoints(model) {
  const waypointsCollection = this.findWaypointsCollection(model);
  if (!waypointsCollection) return;

  waypointsCollection.children.forEach(areaObject => {
    const area = this.createAreaFromObject(areaObject);
    if (!area) return;

    const points = area.extractPoints(areaObject);
    points.forEach(point => {
      this.graph.allPoints.set(point.id, point);
    });
  });
}

```

```

    });
  });
}

findWaypointsCollection(model) {
  let buildingCollection = null;
  model.traverse((object) => {
    if (object.name.toLowerCase() === 'building' && object.children.length > 0) {
      buildingCollection = object;
    }
  });

  if (!buildingCollection) return null;

  let waypointsCollection = null;
  buildingCollection.children.forEach(child => {
    if (child.name === 'WayPoints' && child.children.length > 0) {
      waypointsCollection = child;
    }
  });

  return waypointsCollection;
}

async loadConnections() {
  const areaIndices = this.collectAreaIndices();

  const loadPromises = Array.from(areaIndices).map(areaIndex =>
    fetch(`/models/${this.buildingId}/Area${areaIndex}.txt`)
    .then(response => response.ok ? response.text() : null)
    .then(text => {
      if (text) this.parseConnectionsFile(text);
    })
    .catch(() => {}))
  );
  await Promise.all(loadPromises);
}

parseConnectionsFile(content) {
  content.split('\n').forEach(line => {
    line = line.trim();
    if (!line || line.startsWith('#')) return;

    const [sourceId, targetsStr] = line.split(':').map(s => s.trim());
    if (!sourceId || !targetsStr) return;

    const sourcePoint = this.graph.allPoints.get(sourceId);
    if (!sourcePoint) return;

    targetsStr.split(',').map(id => id.trim()).forEach(targetId => {
      const targetPoint = this.graph.allPoints.get(targetId);
      if (!targetPoint) return;
    });
  });
}

```



```

        sourcePoint.addConnection(targetPoint);
        targetPoint.addConnection(sourcePoint);
    });
});
}
async loadDoorInfo() {
    const response = await fetch(`/models/${this.buildingId}/doors_metadata.json`);
    const doorsData = await response.json();

    for (const [doorId, info] of Object.entries(doorsData)) {
        const doorPoint = this.graph.allPoints.get(doorId);
        if (doorPoint && doorPoint instanceof DoorPoint) {
            doorPoint.setInfo(
                info.number || "",
                info.description_ru || "",
                info.description_en || "",
                this.graph.floorMapping
            );
        }
    }
}
}

```

SceneManager.js (Управление сценой и этажами)

```

async init() {
    const THREE = window.THREE;
    const OrbitControls = window.OrbitControls;

    this.THREE = THREE;

    this.scene = new THREE.Scene();
    this.scene.background = new THREE.Color(0x111122);

    this.camera = new THREE.PerspectiveCamera(
        45,
        this.canvas.clientWidth / this.canvas.clientHeight,
        0.1,
        1000);
    this.camera.position.set(15, 15, 25);
    this.camera.lookAt(0, 0, 0);

    this.renderer = new THREE.WebGLRenderer({ canvas: this.canvas, antialias: true, alpha: false });
    this.renderer.setSize(this.canvas.clientWidth, this.canvas.clientHeight);
    this.renderer.shadowMap.enabled = true;

    this.controls = new OrbitControls(this.camera, this.renderer.domElement);
    this.controls.enableDamping = true;
    this.controls.dampingFactor = 0.05;
    this.controls.maxPolarAngle = Math.PI / 2;

    this.setupLights();
}

```

```

setupLights() {
  const THREE = this.THREE;

  const ambientLight = new THREE.AmbientLight(0x404060);
  this.scene.add(ambientLight);

  const dirLight = new THREE.DirectionalLight(0xff5e6, 0.7);
  dirLight.position.set(4, 7, 3);
  dirLight.castShadow = true;
  this.scene.add(dirLight);
}
analyzeFloors() {
  this.floors.clear();

  let buildingCollection = null;
  this.model.traverse((object) => {
    if (object.name.toLowerCase() === 'building' && object.children.length > 0) {
      buildingCollection = object;
    }
  });

  if (buildingCollection)
    this.findFloorsInCollection(buildingCollection);
}

findFloorsInCollection(collection) {
  const floorRegex = /^floor(?:\d+)/i;

  collection.children.forEach(child => {
    if (child.children && child.children.length > 0) {
      const match = child.name.match(floorRegex);
      if (match) {
        const floorNumber = match[1];
        const meshes = [];
        child.traverse((mesh) => {
          if (mesh.isMesh) {
            if (Array.isArray(mesh.material))
              mesh.material = mesh.material.map(m => m.clone());
            else if (mesh.material)
              mesh.material = mesh.material.clone();
            meshes.push(mesh);
          }
        });
        if (meshes.length > 0)
          this.floors.set(floorNumber, meshes);
      }
    }
  });
}

showFloor(floorNumber) {
  const targetFloor = floorNumber.toString();

```

```

this.floors.forEach((objects, floorKey) => {
  objects.forEach(object => {
    if (floorKey === targetFloor) {
      object.visible = true;
      this.setObjectOpacity(object, 1.0);
    } else {
      if (this.transparentMode) {
        object.visible = true;
        this.setObjectOpacity(object, 0.1);
      } else {
        object.visible = false;
      }
    }
  });
});

if (window.navGraph) {
  const areasOnFloor = window.navGraph.getAreasForFloor(floorNumber);
  window.navGraph.getAllPoints().forEach(point => {
    if (!point.modelObject || !(point instanceof DoorPoint)) return;

    const areaIndex = point.parentArea?.index;
    const isOnCurrentFloor = areasOnFloor.includes(areaIndex);

    if (isOnCurrentFloor) {
      point.modelObject.visible = true;
      this.setObjectOpacity(point.modelObject, 1.0);
    } else {
      if (this.transparentMode) {
        point.modelObject.visible = true;
        this.setObjectOpacity(point.modelObject, 0.1);
      } else {
        point.modelObject.visible = false;
      }
    }
  });
}

this.currentFloor = targetFloor;
}

setObjectOpacity(object, opacity) {
  if (Array.isArray(object.material)) {
    object.material.forEach((material) => {
      if (material) {
        material.transparent = opacity < 1.0;
        material.opacity = opacity;
      }
    });
  } else if (object.material) {
    object.material.transparent = opacity < 1.0;
    object.material.opacity = opacity;
  }
}

```

```

    }
  }
  visualizePath(path) {
    if (!path || path.length < 2 || !window.navGraph || !this.model) return;

    this.clearPath();

    const THREE = this.THREE;
    const points = [];
    const markers = [];

    for (let i = 0; i < path.length; i++) {
      const point = window.navGraph.getPoint(path[i]);
      if (point && point.modelObject) {
        const worldPos = point.modelObject.getWorldPosition(new THREE.Vector3());
        points.push(worldPos.clone());

        if (i === 0 || i === path.length - 1) {
          const sphereMat = new THREE.MeshStandardMaterial({
            color: i === 0 ? 0x44aaff : 0xffaa44,
            emissive: i === 0 ? 0x2266aa : 0xaa6622,
            emissiveIntensity: 0.75
          });
          const sphereGeo = new THREE.SphereGeometry(0.1, 16, 16);
          const sphere = new THREE.Mesh(sphereGeo, sphereMat);
          sphere.position.copy(worldPos);
          this.scene.add(sphere);
          markers.push(sphere);
        }
      }
    }

    const lineGeometry = new THREE.BufferGeometry().setFromPoints(points);
    const lineMaterial = new THREE.LineBasicMaterial({ color: 0x4a90e2 });
    const line = new THREE.Line(lineGeometry, lineMaterial);
    const glowMaterial = new THREE.LineBasicMaterial({
      color: 0x88aaff, transparent: true, opacity: 1
    });
    const glowLine = new THREE.LineSegments(lineGeometry, glowMaterial);

    this.scene.add(glowLine);
    this.scene.add(line);
    this.pathLine = line;
    this.pathGlowLine = glowLine;
    this.pathMarkers = markers;
  }
}

```

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ОРЛОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ И.С. ТУРГЕНЕВА»

УДОСТОВЕРЯЮЩИЙ ЛИСТ № 220885

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

на демонстрационный материал, представленный в электронном виде

Студента Банных Марии Алексеевны шифр 220885
Институт приборостроения, автоматизации и информационных технологий
Кафедра информационных систем и цифровых технологий
Направление 09.03.04 Программная инженерия
Направленность (профиль) Индустриальное производство программного
обеспечения

Наименование документа: Демонстрационные плакаты к выпускной
квалификационной работе

Документ разработал:

Студент Банных М.А. _____

Документ согласован:

Руководитель Ужаринский А.Ю. _____

Нормоконтроль Захарова О.В. _____

Документ утвержден:

И.о. зав. кафедрой Рыженков Д.В. _____

ИНФОРМАЦИОННО-ПОИСКОВАЯ ХАРАКТЕРИСТИКА
ДОКУМЕНТА НА ЭЛЕКТРОННОМ НОСИТЕЛЕ

Наименование		Характеристики документа на электронном носителе
группы атрибутов	атрибута	
1. Описание документа	Обозначение документа (идентификатор(ы) файла(ов))	\\ВКР_Банных_21ПГ.pptx
	Наименование документа	Демонстрационные плакаты к выпускной квалификационной работе
	Класс документа	ЕСКД
	Вид документа	Оригинал документа на электронном носителе
	Аннотация	Демонстрационный материал, отображающий основные этапы выполнения выпускной квалификационной работы
	Использование документа	Операционная система Windows 10, Microsoft PowerPoint 2016
2. Даты и время	Дата и время копирования документа	01.06.2026
	Дата создания документа	19.05.2026
	Дата утверждения документа	21.05.2026
3. Создатели	Автор	Банных М.А.
	Изготовитель	Банных М.А.
4. Внешние ссылки	Ссылки на другие документы	Удостоверяющий лист № 220885
5. Защита	Санкционирование	ОГУ имени И.С. Тургенева
	Классификация защиты	По законодательству РФ
6. Характеристики содержания	Объем информации документа	3795057 Б

7. Структура документа(ов)	Наименование плаката (слайда) №1	Титульный лист
	Наименование плаката (слайда) №2	Цель и задачи работы
	Наименование плаката (слайда) №3	Существующие методы решения проблемы
	Наименование плаката (слайда) №4	Анализ аналогов разрабатываемого решения
	Наименование плаката (слайда) №5	Функциональные требования
	Наименование плаката (слайда) №6	Информационная спецификация
	Наименование плаката (слайда) №7	Поведенческая спецификация
	Наименование плаката (слайда) №8	Структура системы
	Наименование плаката (слайда) №9	Структура модели в Blender
	Наименование плаката (слайда) №10	Схема алгоритма построения маршрута
	Наименование плаката (слайда) №11	Схема алгоритма поиска помещения
	Наименование плаката (слайда) №12	Экранные формы сервиса навигации
	Наименование плаката (слайда) №13	Экранные формы сервиса навигации